



Manual del programador

Sistema de Ascenso de los Bomberos del Estado Miranda

Versión 1.0

Autores:

Cabañales José David

Salcedo Darwin

Sanabria Edgardo

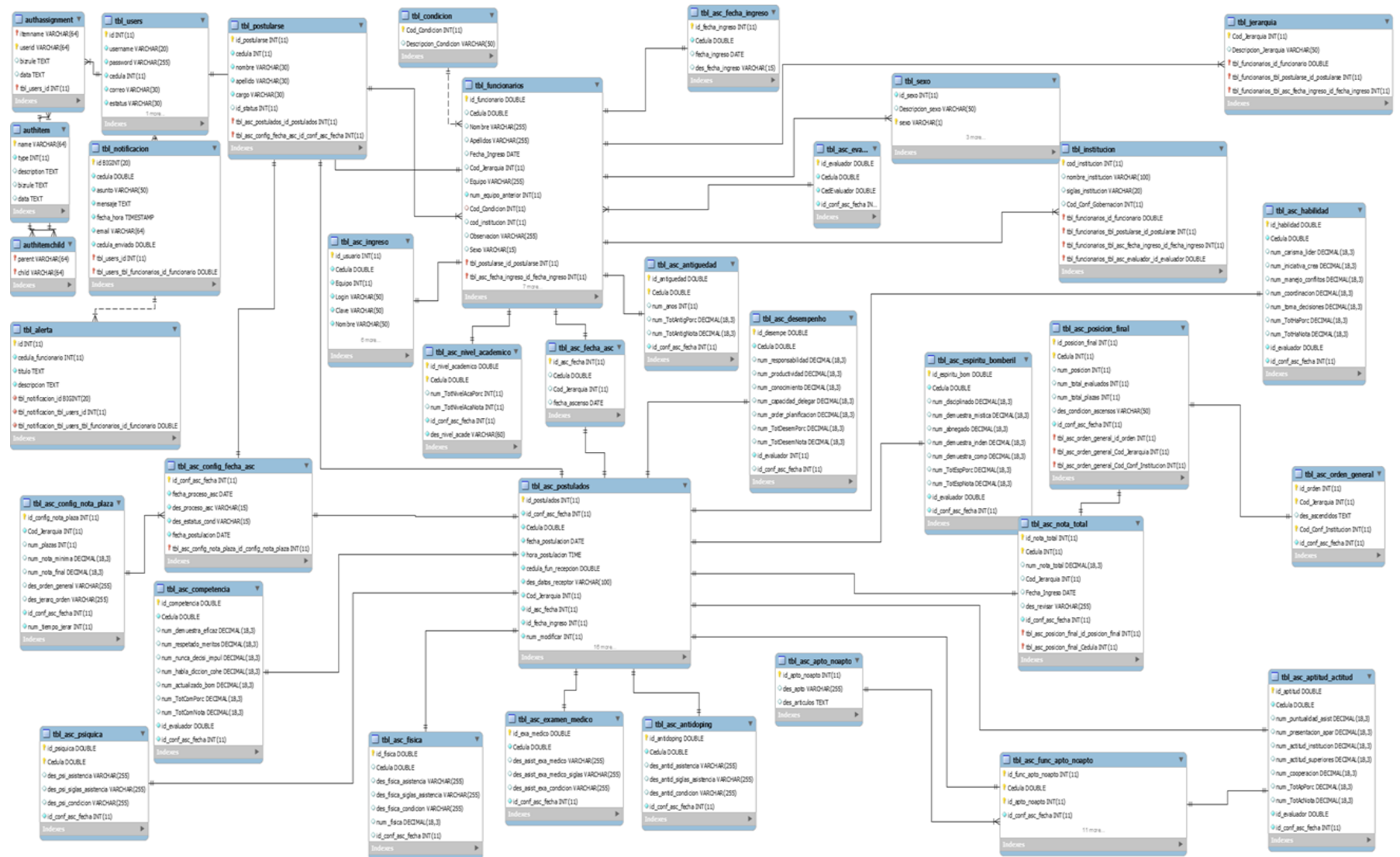
Yriarte Isaac

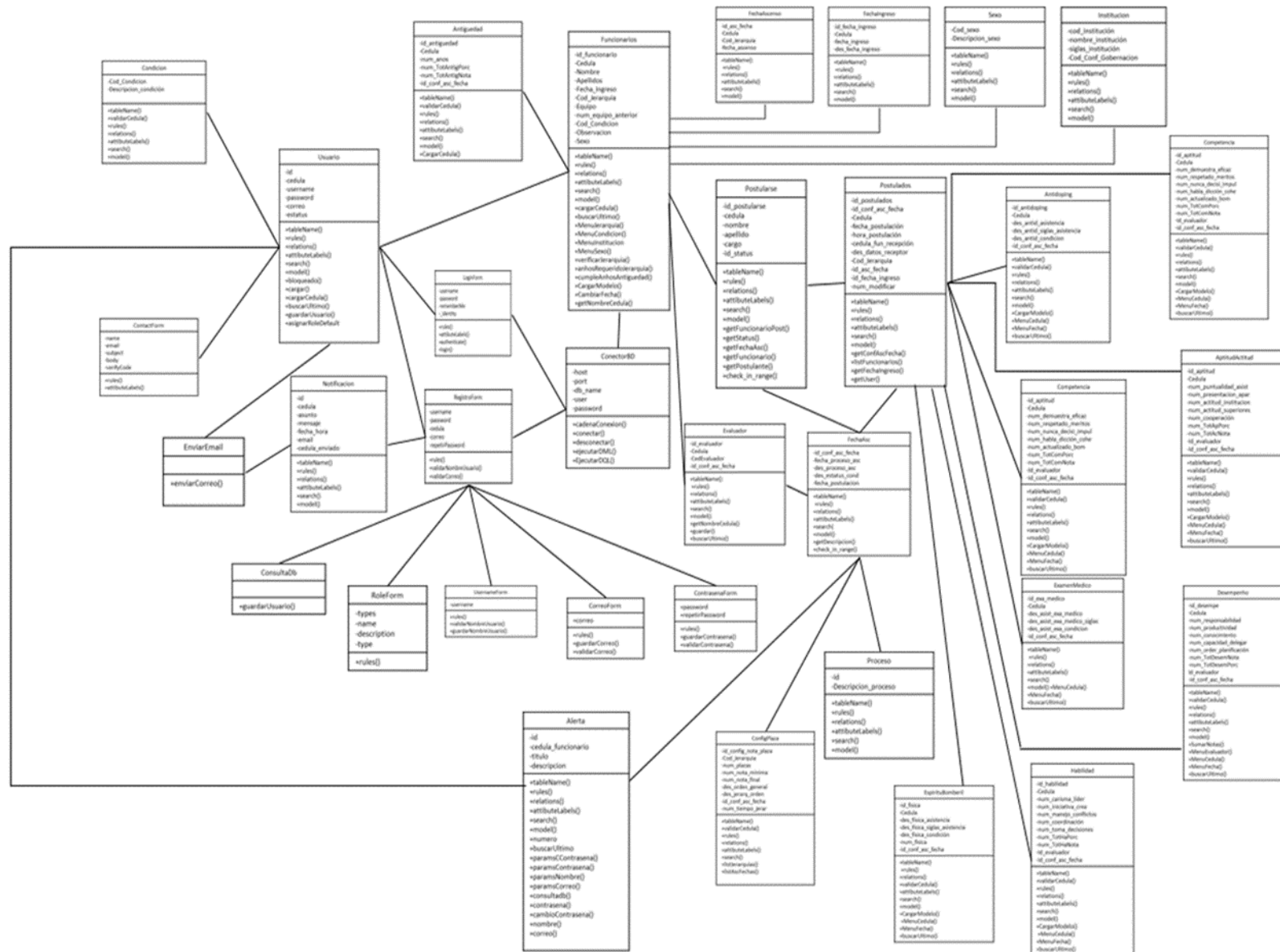
Caracas 2016

Descripción general del sistema

El sistema de ascenso de los bomberos el estado Miranda es un novedoso sistema que permite almacenar, seleccionar y generar rápida y efectivamente respuestas a las solicitudes que se realizan en el proceso de ascenso de los bomberos del estado miranda.

Su fin último es permitir mejorar y automatizar el proceso de ascenso de manera remota al concederle a cada funcionario postularse desde sus casas vía web, facilitando la gestión de los procesos de postulación y ascenso en un mismo sistema.





Diccionario de clases

Clase usuario: Contiene información referente a la persona que usara el sistema estos datos son: Cedula, nombre de usuario, contraseña y correo.

Clase funcionario: Permite contener la permanencia de datos durante la interacción de un usuario luego de haber realizado una búsqueda en las bases de datos.

Clase alerta: Dicha clase se encarga de generar información importante que debe tener en cuenta el usuario para el buen uso de su cuenta.

Clase notificación: Esta clase se encarga de generar de envió de mensajes o notificaciones a él/los correos de los otros usuarios.

Clase contraseñaForm: Clase que pertenece o extiende del usuario, permite la validación de la contraseña en el formulario donde ha sido instanciada.

Clase CorreoForm: Clase que pertenece o extiende del usuario, permite la validación de del correo en el formulario donde ha sido instanciado.

Clase enviarEmail: Se encarga de realizar él envió de correos mediante una configuración preestablecida.

Clase UsernameForm: Clase que pertenece o extiende del usuario, permite la validación del nombre de usuario en el formulario donde ha sido instanciado.

Clase institución: Dicha clase pertenece o extiende del funcionario, permite la validación y relación de las instituciones en el formulario donde ha sido instanciado.

Clase jerarquía: Dicha clase pertenece o extiende del funcionario, permite la validación y relación de la jerarquía en el formulario donde ha sido instanciado.

Clase condición: Dicha clase pertenece o extiende del funcionario, permite la validación y relación de la condición en el formulario donde ha sido instanciado.

Clase Ascenso: Contiene todos los métodos y funciones necesarios para el cálculo de las notas obtenidas en las pruebas por los funcionarios, además de los métodos que permiten listar en forma ordenada cuáles de ellos lograron ascender.

Clase FechaAsc: Contiene los métodos y funciones que necesita el modelo para configurar las fechas de postulación y ascenso para cada proceso de ascenso.

Clase ConfigPlaza: Contiene los métodos y funciones que necesita el modelo para configurar el número de plazas que van a ser requeridas para cada proceso de ascenso.

Clase Postularse: Contiene los métodos y funciones que necesita el modelo para que los funcionarios registrados y activos puedan postularse en el proceso de ascenso que crea la clase FechaAsc.

Clase Postulados: Contiene los métodos y funciones que necesita el modelo para gestionar a todos los postulados que están realizando el proceso de ascenso actual en su momento.

Controlador FechaAsc: Obtiene los datos del administrador que quiere crear una nueva fecha de postulación en el sistema, y renderiza vistas con las fechas de postulación y ascenso que han sido creadas.

Controlador ConfigPlaza: Obtiene los datos del administrador que quiere configurar nuevas plazas para cada jerarquía del proceso de ascenso actual en el sistema, y renderiza vistas con las configuraciones creadas.

Controlador Postularse: Obtiene los datos del usuario que quiere postularse en el proceso de ascenso actual en el sistema, este a su vez renderiza vistas con las la solicitud de postulación, después de pasar por un proceso de validación.

Controlador Postulados: Obtiene los datos del administrador que quiere aceptar a un postulado al proceso de ascenso actual en el sistema, renderiza formularios para asignar evaluadores y carga de notas y finalmente renderiza vistas de los postulados con sus datos cargados.

Controlador usuario: Obtiene los datos de los usuarios que quieren interactuar con el sistema y luego se elige la acción pertinente para mantener su sesión activa, pero antes debe pasar por un tratamiento de autenticación vinculada a la clase.

Controlador Funcionario: Recibe los datos de los funcionarios y se determina, que acción se tomara, luego se procede a enviar la petición a la respectiva vista, finalmente muestra los resultados de la acción.

Controlador Notificación: Recoge la información de un mensaje o notificación, la procesa mediante la clase y posteriormente devuelve el resultado a la vista.

Controlador Alerta: se encarga de enviar las peticiones particulares a la clase alerta para generar la información, posteriormente recibe dichas peticiones ya procesadas y finalmente muestra los datos al usuario.

Controlador de ascenso: Llama a todos los métodos y funciones necesarias para generar la vista (reporte) de todos los funcionarios que ascendieron ordenados por jerarquía

Código Fuente del Sistema

Clase usuario

```
class Usuario extends CActiveRecord
{
    /**
     * @return string the associated database table name
     */
    public function tableName()
    {
        return 'tbl_users';
    }

    /**
     * @return array validation rules for model attributes.
     */
    public function rules()
    {
        // NOTE: you should only define rules for those attributes that
        // will receive user inputs.
        return array(
            array('username, password, correo, estatus', 'required'),
            array('username', 'length', 'max'=>20),
            array('password, correo, estatus', 'length', 'max'=>30),
            // The following rule is used by search().
            // @todo Please remove those attributes that should not be searched.
```

```
        array('id', username, password, correo, estatus , cedula, 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.

    return array(
    );
}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
    return array(
        'id' => 'ID',
        'cedula' => 'Cedula',
        'username' => 'Nombre de Usuario',
        'password' => 'Contraseña',
        'correo' => 'Correo',
        'estatus' => 'Estatus',
    );
}
```

```
/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
 * - Execute this method to get CActiveDataProvider instance which will filter
 *   models according to data in model fields.
 * - Pass data provider to CGridView, CListView or any similar widget.
 *
 * @return CActiveDataProvider the data provider that can return the models
 *   based on the search/filter conditions.
 */
public function search()
{
    // @todo Please modify the following code to remove attributes that should not be
    searched.

    $criteria=new CDbCriteria;

    $criteria->compare('id',$this->id);
    $criteria->compare('username',$this->username,true);
    $criteria->compare('cedula',$this->cedula,true);
    $criteria->compare('password',$this->password,true);
    $criteria->compare('correo',$this->correo,true);
    $criteria->compare('estatus',$this->estatus,true);

    return new CActiveDataProvider($this, array(
        'criteria'=>$criteria,
```

```
    ));  
  
}  
  
/**  
 * Returns the static model of the specified AR class.  
 * Please note that you should have this exact method in all your CActiveRecord descendants!  
 * @param string $className active record class name.  
 * @return Usuario the static model class  
 */  
public static function model($className=__CLASS__)  
{  
    return parent::model($className);  
}  
  
public function bloqueado($id)  
{  
    $sql="SELECT estatus FROM `tbl_users` where id='$id'";  
    $estatus=yii::app()->db->createCommand($sql)->queryAll();  
    if($estatus===null)  
        return False;  
    if ($estatus[0]["estatus"] === 'ACTIVO') return true;  
    else return False;  
}  
  
public static function cargar($id)  
{  
    $model=Usuario::model()->findByPk($id);  
    if($model===null)
```

```
        throw new CHttpException(404,'The requested page does not exist.');
```

```
    return $model;
```

```
}
```

```
public static function cargarCedula($cedula)
```

```
{
```

```
$model= Usuario::model()->find("(cedula)=?",array($cedula));
```

```
    if($model===null)
```

```
    return false;
```

```
    return $model;
```

```
}
```

```
public function buscarUltimo()
```

```
{
```

```
    $sql="SELECT MAX(`id`) FROM `tbl_users`";
```

```
    $num=yii::app()->db->createCommand($sql)->queryAll();
```

```
    if($num===null)
```

```
        throw new CHttpException(404,' no se puede procesar.');
```

```
    return $num[0]["MAX(`id`)"] + 1;
```

```
}
```

```
public function guardarUsuario($username,$password,$cedula,$correo)
```

```
{
```

```
    $conexion=yii::app()->db;
```

```
    $contrasenha=md5($password);
```

```
    $id=$this->buscarUltimo();
```

```
$consulta="insert into tbl_users(id,username,password,cedula,correo,estatus)";  
$consulta.="values ('".$id."','".$username','$contrasenha','$cedula','$correo','ACTIVO')";  
$resultado=$conexion->createCommand($consulta)->execute();  
  
$this->asignarRoleDefault($id);  
  
}  
  
public function asignarRoleDefault($id)  
{  
  
yii::app()->authManager->assign('demo',$id);//asigna el items
```

Clase funcionario

```
<?php  
class Funcionarios extends CActiveRecord  
{  
  
    #public $num_TotNivelAcaNota ;  
    public $des_nivel_acade;  
  
    /**  
     * @return string the associated database table name  
     */  
    public function tableName()  
    {  
        return 'tbl_funcionarios';  
    }  
  
    /**  
     * @return array validation rules for model attributes.  
     */
```

```
public function rules()
{
    // NOTE: you should only define rules for those attributes that
    // will receive user inputs.

    return array(
        array('Cedula, Nombre, Apellidos, Fecha_Ingreso, Cod_Jerarquia, Equipo,
num_categoria, Cod_Condicion, cod_institucion, Sexo , des_nivel_acade', 'required'),
        array('Cod_Jerarquia, num_categoria, Cod_Condicion, cod_institucion',
'numerical', 'integerOnly'=>true),
        array('id_funcionario, Cedula , Equipo,', 'numerical'),
        array('Nombre, Apellidos, Observacion, des_nivel_acade', 'length', 'max'=>255 ,),
        array('Nombre, Apellidos, Observacion, ', 'match', 'pattern'=>'/^[a-z
\\_]+$/i','message'=>'Solo letras '),
        array('Sexo', 'length', 'max'=>15),
        array('Cedula','unique','allowEmpty' => false, 'attributeName' => 'Cedula',
'attributes' => 'Cedula', 'className' => 'Funcionarios', 'caseSensitive' => true , 'skipOnError' => true
,message'=>' Esta cedula existe '),
        array('Equipo','unique','allowEmpty' => false, 'attributeName' => 'Equipo',
'attributes' => 'Equipo', 'className' => 'Funcionarios', 'caseSensitive' => true , 'skipOnError' => true
,message'=>' Esta credencial ya existe '),
        array('num_categoria','validarJerarquia'),
        array('Cedula','numerical',
'integerOnly'=>true,
'min'=>99999999,
'max'=>9999999999,
'tooSmall'=>'No puede ser muy corta',
'tooBig'=>'No existe una cedula con tal digitos'),
        array('Equipo','numerical',
'integerOnly'=>true,
'min'=>1,
'max'=>9999999999,
'tooSmall'=>'No puede ser muy corta',
```

```
'tooBig'=>'No puede pasar de 9 digitos'),

    // The following rule is used by search().

    // @todo Please remove those attributes that should not be searched.

    array('id_funcionario, Cedula, Nombre, Apellidos, Fecha_Ingreso, Cod_Jerarquia,
Equipo, num_categoria, Cod_Condicion, cod_institucion, Observacion, Sexo', 'safe', 'on'=>'search'),

    );

}

public function validarJerarquia($attributes,$params)
{

    #var_dump($this->num_categoria);var_dump($this->Cod_Jerarquia);

    #Yii::app()->end();

    if ($this->num_categoria === '2'){#categoria profesional voluntario

        if($this->Cod_Jerarquia === '0') #si es igual a comandante general

            $this->addError('Cod_Jerarquia','Como profesional voluntario optas hasta
Coronel');

    }

    if ($this->num_categoria === '3'){##categoria asimilado

        if(($this->Cod_Jerarquia === '0')or($this->Cod_Jerarquia === '1')or($this-
>Cod_Jerarquia === '2')) #si es igual a comandante general

            $this->addError('Cod_Jerarquia','Como Asimilado optas hasta Mayor');

    }

}
```

```
/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(
        'genero' => array(self::BELONGS_TO, 'Sexo', 'Sexo'),
        'jerarquia' => array(self::BELONGS_TO, 'Jerarquia', 'Cod_Jerarquia'),
        'institucion' => array(self::BELONGS_TO, 'Institucion', 'cod_institucion'),
        'condicion' => array(self::BELONGS_TO, 'Condicion', 'Cod_Condicion'),
        'Categoria' => array(self::BELONGS_TO, 'Categoria', 'num_categoria'),
        'nivel' => array(self::BELONGS_TO, 'FuncionarioNivelAcademico',
'des_nivel_acade'),

    );
}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
    return array(
        'id_funcionario' => 'Id Funcionario',
        'Cedula' => 'Cedula',
        'Nombre' => 'Nombre',
```

```
'Apellidos' => 'Apellidos',  
'Fecha_Ingreso' => 'Fecha Ingreso',  
'Cod_Jerarquia' => 'Jerarquia',  
'Equipo' => 'Equipo',  
# 'num_equipo_anterior' => 'Num Equipo Anterior',  
'num_categoria' => 'Categoria',  
'Cod_Condicion' => 'Condicion',  
'cod_institucion' => 'Institucion',  
'Observacion' => 'Observacion',  
'Sexo' => 'Sexo',  
# 'num_TotNivelAcaNota' => 'Promedio academico',  
'des_nivel_acade' => 'Nivel academico',  
  
);  
  
}  
  
/**  
 * Retrieves a list of models based on the current search/filter conditions.  
 *  
 * Typical usecase:  
 * - Initialize the model fields with values from filter form.  
 * - Execute this method to get CActiveDataProvider instance which will filter  
 * models according to data in model fields.  
 * - Pass data provider to CGridView, CListView or any similar widget.  
 *  
 * @return CActiveDataProvider the data provider that can return the models  
 * based on the search/filter conditions.  
 */  
  
public function search()
```

```
{  
  
    // @todo Please modify the following code to remove attributes that should not be  
    searched.
```

```
    $criteria=new CDbCriteria;
```

```
    $criteria->compare('id_funcionario',$this->id_funcionario);
```

```
    $criteria->compare('Cedula',$this->Cedula);
```

```
    $criteria->compare('Nombre',$this->Nombre,true);
```

```
    $criteria->compare('Apellidos',$this->Apellidos,true);
```

```
    $criteria->compare('Fecha_Ingreso',$this->Fecha_Ingreso,true);
```

```
    $criteria->compare('Cod_Jerarquia',$this->Cod_Jerarquia);
```

```
    $criteria->compare('Equipo',$this->Equipo,true);
```

```
    #$criteria->compare('num_equipo_anterior',$this->num_equipo_anterior);
```

```
    $criteria->compare('num_categoria',$this->num_categoria);
```

```
    $criteria->compare('Cod_Condicion',$this->Cod_Condicion);
```

```
    $criteria->compare('cod_institucion',$this->cod_institucion);
```

```
    $criteria->compare('Observacion',$this->Observacion,true);
```

```
    $criteria->compare('Sexo',$this->Sexo,true);
```

```
    return new CActiveDataProvider($this, array(  
        'criteria'=>$criteria,
```

```
    ));  
}
```

```
/**
```

```
 * Returns the static model of the specified AR class.
```

```
 * Please note that you should have this exact method in all your CActiveRecord descendants!
```

```
 * @param string $className active record class name.
```

```
* @return Funcionarios the static model class

*/

public static function model($className=__CLASS__)
{
    return parent::model($className);
}

public function cargarCedula($cd)
{
    $model=Funcionarios::model()->find("(cedula)=?",array($cd));

    if($model===null)
        throw new CHttpException(404,' not exist. ');
    return $model;
}

public function buscarUltimo()
{
    $sql="SELECT MAX(`id_funcionario`) FROM `tbl_funcionarios`";
    $num=yii::app()->db->createCommand($sql)->queryAll();
    if($num===null)
        throw new CHttpException(404,' no se puede procesar. ');
    return $num[0]["MAX(`id_funcionario`)"] + 1;
}

public function MenuCategoria()
{
    return CHtml::listData(Categoria::model()->findAll(),'id','Descripcion_categoria');
```

```
}

public function MenuJerarquia()
{
    return CHtml::listData(Jerarquia::model()-
>findAll(),'Cod_Jerarquia','Descripcion_Jerarquia') ;
}

public function MenuCondicion()
{
    return CHtml::listData(Condicion::model()-
>findAll(),'Cod_Condicion','Descripcion_Condicion') ;
}

public function MenuInstitucion()
{
    return CHtml::listData(Institucion::model()-
>findAll(),'cod_institucion','nombre_institucion');
}

public function MenuSexo()
{
    return CHtml::listData(Sexo::model()->findAll(),'sexo','Descripcion_sexo');
}

public function MenuNivel()
{
    return CHtml::listData(Nivelacademico::model()-
>findAll(),'des_nivel_acade','des_nivel_acade');
```

```
}

public function verificarJerarquia()
{
    $jerarquiaAscender=new Jerarquia;

    return $jerarquiaAscender->jerarquiaAscender($this->Cod_Jerarquia);#devuelve la
jerarquia siguiente

}

#devuelve el minimo de años que debo tener para ascender

public function anhosRequeridoJerarquia($numero)#años de jerarquia
{

    $anhos=array(
        '0'=>'3',#1 de coronel a comandante general 3 años
        '1'=>'3',#1 de coronel a comandante general 3 años
        '2'=>'3', # de teniente coronel a coronel 3 años
        '3'=>'3',#de mayor a teniente coronel 3 años
        '4'=>'3', #de capitan a mayor 3 años      (cambie de 2)
        '5'=>'2',#de teniente a capitan 2 años
        '6'=>'2', #de subteniente a teniente 2 años
        '7'=>'2', #de sargento ayudante a subteniente 2 años (cambie de 1)
        '8'=>'1',#de sargento primero a sargento ayudante 1 años
        '9'=>'1', #de sargento segundo a sargento primer 1 año
        '10'=>'1',#de cabo primero a sargento segundo 1 año
        '11'=>'1',#de cabo segundo a cabo primero 1 años
        '12'=>'1',#de distinguido a cabo segundo 1 año
        '13'=>'1',#de bombero a distinguido 1 años
    )
}
```

```
);  
    return $anhos[$numero];  
}
```

```
public function cumpleAnhosAntiguedad($numero,$misAnhos)#años de jerarquia  
{  
    if(($numero)and($misAnhos!=0)){  
        $min=$this->anhosRequeridoJerarquia($numero);  
        if ($misAnhos>$min or $misAnhos=== $min)  
            return true;  
    }  
    return false;  
}
```

```
public static function CargarModelo($cedula)  
{  
    $model= Funcionarios::model()->find("(cedula)=?",array($cedula));  
    if($model===null)  
        return false;  
    #throw new CHttpException(404,' not exist.');
```

```
    return $model;  
}
```

```
public static function CambiarFecha($value)  
{
```

```
list($ano,$mes,$dia)=explode('-', $value);

    $fechaN = $dia.'-'. $mes.'-'. $ano;

    return $fechaN;

}

public function getNombreCedula()
{
    return $this->Nombre.' '. $this->Apellidos.' '. $this->Cedula ;
}

public function getNombreJerarquia($id)
{
    $jerarquia = Jerarquia::model()->findAll('Cod_Jerarquia = '.$id);
    return $jerarquia[0]['Descripcion_Jerarquia'] ;
}

public static function getNivelAcademico($cedula){

    $nivel_aca = Nivelacademicoynota::model()->findAll('Cedula = '.$cedula);
    return $nivel_aca[0]['des_nivel_acade'];

}

public static function getNotaNivelAcademico($cedula){
    $nivel_aca = Nivelacademicoynota::model()->findAll('Cedula = '.$cedula);
    return $nivel_aca[0]['num_TotNivelAcaNota'];

}
```

```
public static function getNombre($cedula){
    $funcionario = Funcionarios::model()->findAll('Cedula = '.$cedula);
    return $funcionario[0]['Nombre'];
}

public static function getApellidos($cedula){
    $funcionario = Funcionarios::model()->findAll('Cedula = '.$cedula);
    return $funcionario[0]['Apellidos'];
}

public static function getNombreApellidos($cedula){
    $nombre_apellidos = array();
    $nombre_apellidos[]= Funcionarios::getNombre($cedula);
    $nombre_apellidos[]= Funcionarios::getApellidos($cedula);
    return $nombre_apellidos;
}
}
```

Clase alerta

```
class Alerta extends CActiveRecord
{
    /**
     * @return string the associated database table name
     */
    public function tableName()
    {
        return 'tbl_alerta';
    }
}
```

```
/**
 * @return array validation rules for model attributes.
 */
public function rules()
{
    // NOTE: you should only define rules for those attributes that
    // will receive user inputs.
    return array(
        array('cedula_funcionario, titulo, descripcion', 'required'),
        array('cedula_funcionario', 'numerical', 'integerOnly'=>true),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be searched.
        array('id, cedula_funcionario, titulo, descripcion', 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(
    );
}

/**
 * @return array customized attribute labels (name=>label)
```

```
*/  
  
public function attributeLabels()  
{  
  
    return array(  
  
        'id' => 'ID',  
  
        'cedula_funcionario' => 'Cedula Funcionario',  
  
        'titulo' => 'Titulo',  
  
        'descripcion' => 'Descripcion',  
  
    );  
  
}  
  
/**  
  
 * Retrieves a list of models based on the current search/filter conditions.  
  
 *  
  
 * Typical usecase:  
  
 * - Initialize the model fields with values from filter form.  
  
 * - Execute this method to get CActiveDataProvider instance which will filter  
 * models according to data in model fields.  
  
 * - Pass data provider to CGridView, CListView or any similar widget.  
  
 *  
  
 * @return CActiveDataProvider the data provider that can return the models  
 * based on the search/filter conditions.  
  
 */  
  
public function search()  
{  
  
    // @todo Please modify the following code to remove attributes that should not be  
searched.  
  
    $criteria=new CDbCriteria;
```

```
$criteria->compare('id',$this->id);

$criteria->compare('cedula_funcionario',$this->cedula_funcionario);

$criteria->compare('titulo',$this->titulo,true);

$criteria->compare('descripcion',$this->descripcion,true);


return new CActiveDataProvider($this, array(

    'criteria'=>$criteria,

));

}

/**

 * Returns the static model of the specified AR class.

 * Please note that you should have this exact method in all your CActiveRecord descendants!

 * @param string $className active record class name.

 * @return Alerta the static model class

 */

public static function model($className=__CLASS__)

{

    return parent::model($className);

}

/**

 *funcion numero simplemente lo que hace es calcular cuantos alertas estan asociadas a un
funcionario

 *de tal forma que retornara un numero ,relacionado a dichas alertas,cuyo fin es hacer visible el
numero

 *alertas en el menu principal.

 */

public function numero()
```

```
{

    $sql=" SELECT COUNT(*) FROM tbl_alerta WHERE cedula_funcionario = ".yii::app()->user-
>getState('cedula').".";

    $estatus=yii::app()->db->createCommand($sql)->queryAll();
    if($estatus===null)
        return False;
    if ($estatus[0]['COUNT(*)']>0) return $estatus[0]['COUNT(*)'];
        else return 0;
}

public function buscarUltimo()
{
    $sql="SELECT MAX(`id`) FROM `tbl_alerta`";
    $num=yii::app()->db->createCommand($sql)->queryAll();
    if($num===null)
        throw new CHttpException(404,' no se puede procesar. ');
    return $num[0]["MAX(`id`)"] + 1;
}

public function paramsCContrasena()
{
    return array('CAMBIO DE CONTRASEÑA','TU CONTRASEÑA HA SIDO CAMBIADA. ');
}

public function paramsContrasena()
{

```

```
        return array('CONTRASEÑA','TU CONTRASEÑA HA SIDO RESTAURADA POR TU NUMERO DE  
CEDULA,POR MOTIVOS DE SEGURIDAD TE RECORDAMOS CAMBIAR TU CONTRASEÑA MAS SEGUIDO.');
```

```
    }
```

```
    public function paramsNombre()
```

```
    {
```

```
        return array('NOMBRE DE USUARIO','TU NOMBRE DE USUARIO HA SIDO CAMBIADO  
RECUERDA QUE PARA EL PROXIMO INICIO DE SESION DEBES TENERLO EN CUENTA.');
```

```
    }
```

```
    public function paramsCorreo()
```

```
    {
```

```
        return array('CORREO','TU CORREO HA SIDO CAMBIADO RECUERDA QUE PARA EL  
PROXIMO INICIO DE SESION DEBES TENERLO EN CUENTA.');
```

```
    }
```

```
    public function paramsAntidoping()
```

```
    {
```

```
        return array('ANTIDOPING','TU RESULTADO DE ANTIDOPING HA SIDO CARGADO EN EL  
SISTEMA.');
```

```
    }
```

```
    public function paramsAptitudActitud()
```

```
    {
```

```
        return array('APTITUD-ACTITUD','TU RESULTADO DE APTITUD-ACTITUD HA SIDO CARGADO  
EN EL SISTEMA.');
```

```
    }
```

```
    public function paramsCompetencia()
```

```
    {
```

```
        return array('COMPETENCIA','TU RESULTADO EN COMPETENCIA HA SIDO CARGADO EN EL
SISTEMA.');
```

```
    }

    public function paramsDesempeno()
    {
        return array('DESEMPEÑO','TU RESULTADO EN DESEMPEÑO HA SIDO CARGADO EN EL
SISTEMA.');
```

```
    }

    public function paramsEspirituBomberil()
    {
        return array('ESPIRITU BOMBERIL','TU RESULTADO EN ESPIRITU BOMBERIL HA SIDO
CARGADO EN EL SISTEMA.');
```

```
    }

    public function paramsExamenMedico()
    {
        return array('EXAMEN MEDICO','TU RESULTADO EN EXAMEN MEDICO HA SIDO CARGADO
EN EL SISTEMA.');
```

```
    }

    public function paramsFisica()
    {
        return array('EXAMEN FISICO','TU RESULTADO EN EXAMEN FISICO HA SIDO CARGADO EN
EL SISTEMA.');
```

```
    }

    public function paramsHabilidad()
    {
        return array('HABILIDAD','TU RESULTADO EN HABILIDADES HA SIDO CARGADO EN EL
SISTEMA.');
```

```
    }

    public function CONSULTADB($param,$cedula = null)
```

```
{  
  
    if (empty($cedula)) {  
  
        $cedula=yii::app()->user->getState('cedula');  
  
    }  
  
    $id=$this->buscarUltimo();  
  
    $sql="INSERT INTO tbl_alerta (`id`, `cedula_funcionario`, `titulo`, `descripcion`)";  
    $sql .="values ($id, '$cedula', ".$param.".)";  
  
    $conexion=yii::app()->db;  
    $resultado=$conexion->createCommand($sql)->execute();  
}  
  
public function contrasena()  
{  
    $paramametros=$this->paramsContrasena();  
    $sql =""$paramametros[0]', '$paramametros[1]"";  
    $this->CONSULTADB($sql);  
}  
  
public function cambioContrasena()  
{  
    $paramametros=$this->paramsCContrasena();  
    $sql =""$paramametros[0]', '$paramametros[1]"";  
    $this->CONSULTADB($sql);  
}  
  
public function nombre()  
{  
    $paramametros=$this->paramsNombre();  
    $sql =""$paramametros[0]', '$paramametros[1]"";  
    $this->CONSULTADB($sql);
```

```
}
```

```
        public function correo()
    {
        $paramametros=$this->paramsCorreo();
        $sql =""$paramametros[0]', '$paramametros[1]'';
        $this->CONSULTADB($sql);
    }
```

```
        public function Generar($caso,$cedula)
    {
        switch ($caso)
        {
            case 'Antidoping':
                $paramametros=$this->paramsAntidoping();
                break;

            case 'EspirituBomberil':
                $paramametros=$this->paramsEspirituBomberil();
                break;

            case 'Fisica':
                $paramametros=$this->paramsFisica();
                break;

            case 'Habilidad':
                $paramametros=$this->paramsHabilidad();
                break;

            case 'AptitudA':
```

```
        $paramametros=$this->paramsAptitudActitud();  
        break;  
  
        case 'Desempeno':  
            $paramametros=$this->paramsDesempeno();  
            break;  
  
        case 'Competencia':  
            $paramametros=$this->paramsCompetencia();  
            break;  
  
        case 'ExamenMedico':  
            $paramametros=$this->paramsExamenMedico();  
            break;  
  
    }  
  
    $sql = "$paramametros[0]', '$paramametros[1]";  
    $this->CONSULTADB($sql,$cedula);  
}  
  
}
```

Clase notificacion

```
class Notificacion extends CActiveRecord  
{  
    /**
```

```
* @return string the associated database table name
*/

public function tableName()
{
    return 'tbl_notificacion';
}

/**
 * @return array validation rules for model attributes.
 */
public function rules()
{
    // NOTE: you should only define rules for those attributes that
    // will receive user inputs.
    return array(
        array('asunto, mensaje, email', 'required'),
        array('cedula, cedula_enviado', 'numerical'),
        array('asunto', 'length', 'max'=>50),
        array('email', 'length', 'max'=>64),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be searched.
        array('id, cedula, asunto, mensaje, fecha_hora, email, cedula_enviado', 'safe',
'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
```

```
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(

    );
}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
    return array(
        'id' => 'ID',
        'cedula' => 'Cedula',
        'asunto' => 'Asunto',
        'mensaje' => 'Mensaje',
        'fecha_hora' => 'Fecha Hora',
        'email' => 'Email',
        'cedula_enviado' => 'Cedula Enviado',
    );
}

/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
```

```
* - Execute this method to get CActiveDataProvider instance which will filter
* models according to data in model fields.
* - Pass data provider to CGridView, CListView or any similar widget.
*
* @return CActiveDataProvider the data provider that can return the models
* based on the search/filter conditions.
*/
public function search()
{
    // @todo Please modify the following code to remove attributes that should not be
searched.

    $criteria=new CDbCriteria;

    $criteria->compare('id',$this->id,true);
    $criteria->compare('cedula',$this->cedula);
    $criteria->compare('asunto',$this->asunto,true);
    $criteria->compare('mensaje',$this->mensaje,true);
    $criteria->compare('fecha_hora',$this->fecha_hora,true);
    $criteria->compare('email',$this->email,true);
    $criteria->compare('cedula_enviado',$this->cedula_enviado);

    return new CActiveDataProvider($this, array(
        'criteria'=>$criteria,
    ));
}

/**
 * Returns the static model of the specified AR class.
```

* Please note that you should have this exact method in all your CActiveRecord descendants!

* @param string \$className active record class name.

* @return Notificacion the static model class

*/

```
public static function model($className=__CLASS__)
```

```
{
```

```
    return parent::model($className);
```

```
}
```

```
}
```

Clase contraseñaForm:

```
<?php
```

```
/**
```

```
*
```

```
*/
```

```
class ContraseñaForm extends CFormModel
```

```
{
```

```
    public $passActual;
```

```
    public $password;
```

```
    public $repetirPassword;
```

```
/**
```

```
    * Declares the validation rules.
```

```
*/
```

```
    public function rules()
```

```
{
```

```
return array(

    array('passActual,password,repertirPassword', 'required'),

    #array('password,passActual ', 'match','pattern'=>'/^[a-z0-9]+$/', 'message'=>'Solo
numeros y letras'),

    array('password','length','min'=>8,'tooShort'=>'La contraseña debe tener minimo
8 caracteres'),

    array('repertirPassword', 'compare','compareAttribute'=>'password','message'=>'
La contraseña no coincide '),),

    array('passActual', 'ValidarActual' ),

);

}
```

```
public function validarActual($attributes,$params)

{

    $conexion=yii::app()->db;

    $sql="SELECT password FROM tbl_users WHERE `cedula` = '".yii::app()->user-
>getState('cedula')."';";

    $resultado=$conexion->createCommand($sql);

    $filas=$resultado->query();

    $pass=strtolower(md5($this->passActual));

    foreach($filas as $fila)

    {

        if ($pass !== strtolower($fila['password']))

        {

            $this->addError('passActual','Contraseña no valida');

            break;

        }

    }

}
```

```
}
```

```
}
```

```
public function validarContrasena()
{
    $pass=md5($password);
    $conexion=yii::app()->db;
    $sql="UPDATE `tbl_users` SET `password`='".$.$pass.'" WHERE `cedula` = '".yii::app()->user-
>getState('cedula')."';";
    $resultado=$conexion->createCommand($sql);
    $resultado->query();
}
```

```
public function guardarContrasena($password)
{
    $pass=md5($password);
    $conexion=yii::app()->db;
    $sql="UPDATE `tbl_users` SET `password`='".$.$pass.'" WHERE `cedula` = '".yii::app()->user-
>getState('cedula')."';";
    $resultado=$conexion->createCommand($sql);
    $resultado->query();
}
```

```
        public function attributeLabels()
        {
            return array(
                'passActual' => 'Contraseña actual',
                'password' => 'Contraseña nueva',
                'repetirPassword' => 'Repita contraseña',

            );
        }
    }
```

Clase CorreoForm:

```
<?php

/**
 *
 */
class CorreoForm extends CFormModel
{

    public $correo;

    /**
     * Declares the validation rules.
     */
}
```

```
public function rules()
{
    return array(

        array('correo', 'required'),
        array('correo', 'email', 'message'=>'Formato del correo esta incorrecto '),
        array('correo', 'validarCorreo'),

    );
}

public function validarCorreo($attributes,$params)
{
    $conexion=yii::app()->db;
    $sql="SELECT correo FROM tbl_users WHERE correo = '". $this->correo. "'";
    $resultado=$conexion->createCommand($sql);
    $filas=$resultado->query();

    foreach($filas as $fila)
    {
        if ($this->correo === $fila['correo'])
        {
            $this->addError('correo','Nombre de correo no disponible');

            break;
        }
    }
}
```

```
$mail->Host = 'smtp.mandrillapp.com';           // Specify main and
backup server

$mail->Port = 587;                               // Set the SMTP port

$mail->SMTPAuth = true;                          // Enable SMTP authentication

$mail->Username = 'darwin.bsd@hotmail.com';      // SMTP
username

$mail->Password = 'iybi_4sulGcoHvHpsA1Hlg';     // SMTP
password

$mail->SMTPSecure = 'tls';                      // Enable encryption, 'ssl' also
accepted

$mail->From = 'sabem@gmail.com';

$mail->FromName = 'SISTEMA DE ASCENSO DE LOS BOMBEROS DEL
ESTADO MIRANDA';

$mail->AddAddress('darwin.bsd@hotmail.com'); // Add a recipient
$mail->AddAddress($email);                  // Name is optional

$mail->IsHTML(true);                          // Set email format to HTML

$mail->Subject = $asunto;

$mail->Body = '<strong>'.$mensaje.'</strong>';

$mail->AltBody = 'This is the body in plain text for non-HTML mail clients';

if(!$mail->Send()) {
    #echo 'Message could not be sent.';
    #echo 'Mailer Error: ' . $mail->ErrorInfo;
    # Yii::app()->user->setFlash('error','Mailer Error: ' . $mail->ErrorInfo);
    return false;
} else
    return true;#Yii::app()->user-
>setFlash('success','Enviado ');
```

```
}
```

```
}
```

```
?>
```

Clase UsernameForm:

```
<?php
```

```
/**
```

```
*
```

```
*/
```

```
class UsernameForm extends CFormModel
```

```
{
```

```
    public $username;
```

```
/**
```

```
 * Declares the validation rules.
```

```
*/
```

```
    public function rules()
```

```
    {
```

```
        return array(
```

```
            array('username', 'required'),
```

```
array('username', 'match', 'pattern'=>'/^[a-z0-9áéíóúàèìòùäëïöüñ_]+$/i', 'message'=>'Solo numeros, letras y guion bajo '),  
  
array('username', 'validarNombreUsuario'),  
  
);  
  
}  
  
public function validarNombreUsuario($attributes,$params)  
{  
  
    $conexion=yii::app()->db;  
  
    $sql="SELECT username FROM tbl_users WHERE username = '".$this->username."'";  
  
    $resultado=$conexion->createCommand($sql);  
  
    $filas=$resultado->query();  
  
    foreach($filas as $fila)  
    {  
  
        if ($this->username === $fila['username'])  
        {  
  
            $this->addError('username', 'Nombre de usuario no disponible');  
  
            break;  
  
        }  
  
    }  
  
}  
  
  
public function guardarNombreUsuario()  
{  
  
    $conexion=yii::app()->db;  
  
    $consulta="UPDATE `tbl_users` SET `username`='".$this->username."' WHERE `cedula` =  
    '".yii::app()->user->getState('cedula')."'";  
  
    $resultado=$conexion->createCommand($consulta)->execute();
```

```
}
```

```
}
```

Clase institución:

```
<?php
```

```
/**
```

```
 * This is the model class for table "tbl_institucion".
```

```
 *
```

```
 * The followings are the available columns in table 'tbl_institucion':
```

```
 * @property integer $cod_institucion
```

```
 * @property string $nombre_institucion
```

```
 * @property string $siglas_institucion
```

```
 * @property integer $Cod_Conf_Gobernacion
```

```
 */
```

```
class Institucion extends CActiveRecord
```

```
{
```

```
    /**
```

```
     * @return string the associated database table name
```

```
    */
```

```
    public function tableName()
```

```
    {
```

```
        return 'tbl_institucion';
```

```
    }
```

```
    /**
```

```
     * @return array validation rules for model attributes.
```

```
    */
```

```
public function rules()
{
    // NOTE: you should only define rules for those attributes that
    // will receive user inputs.
    return array(
        array('cod_institucion', 'required'),
        array('cod_institucion, Cod_Conf_Gobernacion', 'numerical', 'integerOnly'=>true),
        array('nombre_institucion', 'length', 'max'=>100),
        array('siglas_institucion', 'length', 'max'=>20),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be searched.
        array('cod_institucion, nombre_institucion, siglas_institucion,
Cod_Conf_Gobernacion', 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(
    );
}

/**
 * @return array customized attribute labels (name=>label)
```

```
*/  
  
public function attributeLabels()  
{  
  
    return array(  
  
        'cod_institucion' => 'Cod Institucion',  
  
        'nombre_institucion' => 'Nombre Institucion',  
  
        'siglas_institucion' => 'Siglas Institucion',  
  
        'Cod_Conf_Gobernacion' => 'Cod Conf Gobernacion',  
  
    );  
  
}  
  
/**  
  
 * Retrieves a list of models based on the current search/filter conditions.  
  
 *  
  
 * Typical usecase:  
  
 * - Initialize the model fields with values from filter form.  
  
 * - Execute this method to get CActiveDataProvider instance which will filter  
 * models according to data in model fields.  
  
 * - Pass data provider to CGridView, CListView or any similar widget.  
  
 *  
 * @return CActiveDataProvider the data provider that can return the models  
 * based on the search/filter conditions.  
  
 */  
  
public function search()  
{  
  
    // @todo Please modify the following code to remove attributes that should not be  
searched.  
  
    $criteria=new CDbCriteria;
```

```
$criteria->compare('cod_institucion',$this->cod_institucion);

$criteria->compare('nombre_institucion',$this->nombre_institucion,true);

$criteria->compare('siglas_institucion',$this->siglas_institucion,true);

$criteria->compare('Cod_Conf_Gobernacion',$this->Cod_Conf_Gobernacion);


return new CActiveDataProvider($this, array(
    'criteria'=>$criteria,
));
}

/**
 * Returns the static model of the specified AR class.
 *
 * Please note that you should have this exact method in all your CActiveRecord descendants!
 * @param string $className active record class name.
 * @return Institucion the static model class
 */
public static function model($className=__CLASS__)
{
    return parent::model($className);
}
}
```

Clase jerarquía:

```
<?php

class Jerarquia extends CActiveRecord
{
    /**
     * @return string the associated database table name
     */
}
```

```
*/  
  
public function tableName()  
{  
  
    return 'tbl_jerarquia';  
  
}  
  
/**  
 * @return array validation rules for model attributes.  
 */  
  
public function rules()  
{  
  
    // NOTE: you should only define rules for those attributes that  
    // will receive user inputs.  
  
    return array(  
  
        array('Cod_Jerarquia', 'required'),  
        array('Cod_Jerarquia', 'numerical', 'integerOnly'=>true),  
        array('Descripcion_Jerarquia', 'length', 'max'=>50),  
  
        // The following rule is used by search().  
        // @todo Please remove those attributes that should not be searched.  
        array('Cod_Jerarquia, Descripcion_Jerarquia', 'safe', 'on'=>'search'),  
  
    );  
  
}  
  
/**  
 * @return array relational rules.  
 */  
  
public function relations()  
{  
  
    // NOTE: you may need to adjust the relation name and the related
```

```
// class name for the relations automatically generated below.

return array(

    'tblFuncionarios' => array(self::HAS_MANY, 'TblFuncionarios', 'Cod_Jerarquia'),
    'codJerarquia' => array(self::BELONGS_TO, 'TblFuncionarios', 'Cod_Jerarquia'),

);

}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
    return array(
        'Cod_Jerarquia' => 'Cod Jerarquia',
        'Descripcion_Jerarquia' => 'Descripcion Jerarquia',
    );
}

/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
 * - Execute this method to get CActiveDataProvider instance which will filter
 *   models according to data in model fields.
 * - Pass data provider to CGridView, CListView or any similar widget.
 *
 * @return CActiveDataProvider the data provider that can return the models
 *   based on the search/filter conditions.
```

```
*/  
  
public function search()  
{  
  
    // @todo Please modify the following code to remove attributes that should not be  
searched.  
  
    $criteria=new CDbCriteria;  
  
    $criteria->compare('Cod_Jerarquia',$this->Cod_Jerarquia);  
    $criteria->compare('Descripcion_Jerarquia',$this->Descripcion_Jerarquia,true);  
  
    return new CActiveDataProvider($this, array(  
        'criteria'=>$criteria,  
    ));  
}  
  
/**  
 * Returns the static model of the specified AR class.  
 * Please note that you should have this exact method in all your CActiveRecord descendants!  
 * @param string $className active record class name.  
 * @return Jerarquia the static model class  
 */  
  
public static function model($className=__CLASS__)  
{  
    return parent::model($className);  
}  
  
public function jerarquiaAscender($codigo)  
{
```

```
$jerarquia=new Jerarquia;

$numero=$codigo - 1;

if ($numero >= 0)
{
    $resultado=$jerarquia->find("(Cod_Jerarquia)=?",array($numero));

}

else

    return false;

if ($resultado != null ) return $resultado->Cod_Jerarquia;
else return false;

}

}
```

Clase condición:

```
<?php

/**
 * This is the model class for table "tbl_condicion".
 *
 * The followings are the available columns in table 'tbl_condicion':
 * @property integer $Cod_Condicion
 * @property string $Descripcion_Condicion
 */
class Condicion extends CActiveRecord
{
```

```
/**
 * @return string the associated database table name
 */
public function tableName()
{
    return 'tbl_condicion';
}

/**
 * @return array validation rules for model attributes.
 */
public function rules()
{
    // NOTE: you should only define rules for those attributes that
    // will receive user inputs.
    return array(
        array('Cod_Condicion', 'numerical', 'integerOnly'=>true),
        array('Descripcion_Condicion', 'length', 'max'=>50),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be searched.
        array('Cod_Condicion, Descripcion_Condicion', 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{

```

```
// NOTE: you may need to adjust the relation name and the related
// class name for the relations automatically generated below.

return array(

);

}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
    return array(
        'Cod_Condicion' => 'Cod Condicion',
        'Descripcion_Condicion' => 'Descripcion Condicion',
    );
}

/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
 * - Execute this method to get CActiveDataProvider instance which will filter
 *   models according to data in model fields.
 * - Pass data provider to CGridView, CListView or any similar widget.
 *
 * @return CActiveDataProvider the data provider that can return the models
 *   based on the search/filter conditions.
 */
```

```
public function search()
{
    // @todo Please modify the following code to remove attributes that should not be
    searched.

    $criteria=new CDbCriteria;

    $criteria->compare('Cod_Condicion',$this->Cod_Condicion);
    $criteria->compare('Descripcion_Condicion',$this->Descripcion_Condicion,true);

    return new CActiveDataProvider($this, array(
        'criteria'=>$criteria,
    ));
}

/**
 * Returns the static model of the specified AR class.
 * Please note that you should have this exact method in all your CActiveRecord descendants!
 * @param string $className active record class name.
 * @return Condicion the static model class
 */
public static function model($className=__CLASS__)
{
    return parent::model($className);
}
}
```

Controlador usuario:

<?php

class UsuarioController extends Controller

```
{  
  
    /**  
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning  
     * using two-column layout. See 'protected/views/layouts/column2.php'.  
     */  
    public $layout='//layouts/column2';  
  
    /**  
     * @return array action filters  
     */  
    public function filters()  
    {  
        return array(  
            'accessControl', // perform access control for CRUD operations  
            'postOnly + delete', // we only allow deletion via POST request  
        );  
    }  
  
    /**  
     * Specifies the access control rules.  
     * This method is used by the 'accessControl' filter.  
     * @return array access control rules  
     */  
    public function accessRules()  
    {  
        return array(  
            array('allow', // allow all users to perform 'index' and 'view' actions  
                'actions'=>array('index','view','obtenerEtiqueta','AjaxActivador',
```

```
'ReiniciarPass','cambiarContrasena'),
    'roles'=>array('admin'),
),
array('allow', // allow authenticated user to perform 'create' and 'update' actions
    'actions'=>array('create','update','cambiarContrasena','datos'),
    'users'=>array('@'),
),
array('allow', // allow admin user to perform 'admin' and 'delete' actions
    'actions'=>array('admin','delete','asignarPermiso','Bloquear','username'),
    'roles'=>array('admin'),
),
array('deny', // deny all users
    'users'=>array('*'),
),
);
}

/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $role=new RoleForm;

    if (isset($_POST["RoleForm"]))
    {
        $role->attributes=$_POST["RoleForm"];
        if($role->validate())
```

```
{  
  
    yii::app()->authManager->createRole($role->name,$role->description);  
    yii::app()->authManager->assign($role->name,$id);  
    $this->redirect(array("view","id"=>$id));  
  
}  
  
}  
  
$this->render('view',array(  
    'model'=>$this->loadModel($id),  
    'role'=>$role,//envio mi formulario  
  
));  
  
}  
  
/**  
 * Creates a new model.  
 * If creation is successful, the browser will be redirected to the 'view' page.  
 */  
public function actionCreate()  
{  
  
    $model=new Usuario;  
  
    // Uncomment the following line if AJAX validation is needed  
    // $this->performAjaxValidation($model);  
  
    if(isset($_POST['Usuario']))  
    {  
  
        $model->attributes=$_POST['Usuario'];  
        if($model->save())  
            $this->redirect(array('view','id'=>$model->id));  
    }  
}
```

```
}

$this->render('create',array(

    'model'=>$model,

));

}

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */
public function actionUpdate($id)
{
    $model=$this->loadModel($id);

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Usuario']))
    {
        $model->attributes=$_POST['Usuario'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id));
    }

    $this->render('update',array(

        'model'=>$model,

    ));
}
```

```
}

/**
 * Deletes a particular model.
 * If deletion is successful, the browser will be redirected to the 'admin' page.
 * @param integer $id the ID of the model to be deleted
 */
public function actionDelete($id)
{
    $this->loadModel($id)->delete();

    // if AJAX request (triggered by deletion via admin grid view), we should not redirect the
    browser
    if(!isset($_GET['ajax']))
        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
}

/**
 * Lists all models.
 */
public function actionIndex(){

    $dataProvider=new CActiveDataProvider('Usuario');
    $this->render('index',array(
        'dataProvider'=>$dataProvider,
    ));
}

/**
```

* Manages all models.

*/

public function actionAdmin()

{

 \$model=new Usuario('search');

 \$model->unsetAttributes(); // clear any default values

 if(isset(\$_GET['Usuario']))

 \$model->attributes=\$_GET['Usuario'];

 \$this->render('admin',array(

 'model'=>\$model,

));

}

public function actionAsignarPermiso(\$id)

{

 foreach(Yii::app()->authManager->getAuthItems() as \$data)

 {

 if (\$data->name==\$_GET["item"]){

 if(Yii::app()->authManager->checkAccess(\$data->name,\$id)) //si

llega el item lo elimina

 Yii::app()->authManager->

revoke(\$_GET["item"],\$id);//lo quita

 else

 Yii::app()->authManager->

assign(\$_GET["item"],\$id);//asigna el items

 }else{

```
        #if(yii::app()->authManager->checkAccess($data->name,$id))//si llega el item lo  
elimina  
  
        yii::app()->authManager->revoke($data->name,$id);//lo quita
```

```
    }  
}  
  
    $this->redirect(array("view","id"=>$id));  
}  
  
/**  
 * Returns the data model based on the primary key given in the GET variable.  
 * If the data model is not found, an HTTP exception will be raised.  
 * @param integer $id the ID of the model to be loaded  
 * @return Usuario the loaded model  
 * @throws CHttpException  
 */  
public function loadModel($id)  
{  
    $model=Usuario::model()->findByPk($id);  
    if($model===null)  
        throw new CHttpException(404,'The requested page does not exist.');
```

```
    return $model;  
}  
  
/**  
 * Performs the AJAX validation.  
 * @param Usuario $model the model to be validated  
 */  
protected function performAjaxValidation($model)
```

```
{  
  
    if(isset($_POST['ajax']) && $_POST['ajax']==='usuario-form')  
    {  
  
        echo CActiveForm::validate($model);  
  
        Yii::app()->end();  
    }  
}  
  
// en la controladora o componente:  
public function obtenerEtiqueta($model){  
  
    #var_dump($model->attributes['estatus']);  
  
    if(($model->attributes['estatus']=='ACTIVO') or ($model->  
attributes['estatus']=='ACTIVO')){  
  
        return "Desactivado";  
    }else{  
  
        return "Activado";  
    }  
}  
  
public function actionAjaxActivador($id){  
  
    // activara al usuario en la sesion de ejemplo, pudimos haber dicho aqui:  
  
    // "por favor activa este usuario en la nomina seleccionada"  
  
  
    $etiqueta = "";  
    $model=Usuario::model();  
    $aux=Usuario::model();  
    $aux=$model::cargar($id);  
    if ($aux->attributes['estatus']=='ACTIVO')#activo  
    {
```

```
$aux->updateByPk($id,array('estatus'=>'INACTIVO'));

$etiqueta = "Activado";

}

else {

    $aux->updateByPk($id,array('estatus'=>'ACTIVO'));

    $etiqueta = "Desactivado";

}

echo $etiqueta;

}
```

```
public function actionBloquear($id)
{
    $model=Usuario::model();

    $aux=Usuario::model();

    $aux=$model::cargar($id);

    if ($aux->attributes['estatus']=='ACTIVO')#activo
    {
        $aux->updateByPk($id,array('estatus'=>'INACTIVO'));
    }

    else $aux->updateByPk($id,array('estatus'=>'ACTIVO'));

    $this->redirect(array("view","id"=>$id));

}
```

```
public function actionDatos(){
```

```
$Username= new UsernameForm();
```

```
$Correo= new CorreoForm();
```

```
if (isset($_POST['ajax']) && $_POST['ajax'] === 'UsernameForm' )
{
    echo CActiveForm::validate($Username);
    yii::app()->end();
}
```

```
if (isset($_POST['ajax']) && $_POST['ajax'] === 'CorreoForm' )
{
    echo CActiveForm::validate($Correo);
    yii::app()->end();
}
```

```
if(isset($_POST['UsernameForm']))
{
    $Username->attributes=$_POST['UsernameForm'];
    $Username->guardarNombreUsuario();
    $alerta= new Alerta();
    $alerta->nombre();
    Yii::app()->user->setFlash('success','Nombre de usuario actualizado
satisfactoriamente');
}
```

```
if(isset($_POST['CorreoForm']))
{
    $Correo->attributes=$_POST['CorreoForm'];
    $Correo->guardarCorreo();
    $alerta= new Alerta();
    $alerta->correo();
    Yii::app()->user->setFlash('success','Correo actualizado satisfactoriamente');
}

$this->render('Datos',array('Username'=>$Username,'Correo'=>$Correo,'datos'=>$d=Usuario::model()->cargarCedula(Yii::app()->user->getState('cedula'))));
}

public function actionCambiarContraseña(){
    $model= new ContraseñaForm();
    $alerta= new Alerta();
    $msg='La contraseña ha sido cambiada';
    $usuario=Usuario::model()->cargarCedula(Yii::app()->user->getState('cedula'));

    if (isset($_POST['ajax']) && $_POST['ajax'] === 'ContraseñaForm' )
    {
        echo CActiveForm::validate($model);
        Yii::app()->end();
    }

    if(isset($_POST['ContraseñaForm']))
    {
        $model->attributes=$_POST['ContraseñaForm'];
```

```
$model->guardarContrasena($model->attributes['password']);

$alerta->cambioContrasena();

#$this->refresh();

$model->passActual="";$model->password="";

$model->repetirPassword="";

Yii::app()->user->setFlash('success',$msg );

}

    $this->render('Contrasena',
    array('model'=>$model,'usuario'=>$usuario));

}

public function actionReiniciarPass($id)
{

    $model=Usuario::model();
    $aux=Usuario::model();
    $aux=$model::cargar($id);

    $msg='La contraseña de "'.$aux->username $aux->cedula".' ha sido reiniciada
satisfactoriamente ';
```

```
    $contrasena=md5($aux->cedula);
    $aux->updateByPk($aux->id,array('password'=>$contrasena));

    Yii::app()->user->setFlash('notice',$msg );

    $this->actionAdmin();

    #bandera

    #enviar notificacion usuario

    #bandera para el usurio que cambien su clave
```

```
}
```

```
public function actionusername(){
```

```
$model= new DatosForm();
```

```
$msg='tus Datos han sido actualizados';
```

```
if (isset($_POST['ajax']) && $_POST['ajax'] === 'DatosForm' )
```

```
{
```

```
    echo CActiveForm::validate($model);
```

```
    yii::app()->end();
```

```
}
```

```
if(isset($_POST['DatosForm']))
```

```
{
```

```
    $model->attributes=$_POST['DatosForm'];
```

```
    #$model->guardarContrasena($model->attributes['password']);
```

```
    Yii::app()->user->setFlash('success',$msg );
```

```
}
```

```
    $this->render('_datos',array('model'=>$model,'datos'=>$d=Usuario::model()-  
>cargarCedula(yii::app()->user->getState('cedula'))));
```

```
}
```

```
}
```

Controlador Funcionario:

<?php

```
class FuncionariosController extends Controller
```

```
{
```

```
    /**
```

```
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning
```

```
     * using two-column layout. See 'protected/views/layouts/column2.php'.
```

```
     */
```

```
    public $layout='//layouts/column2';
```

```
    /**
```

```
     * @return array action filters
```

```
     */
```

```
    public function filters()
```

```
    {
```

```
        return array(
```

```
            'accessControl', // perform access control for CRUD operations
```

```
            'postOnly + delete', // we only allow deletion via POST request
```

```
        );
```

```
    }
```

```
    /**
```

```
     * Specifies the access control rules.
```

```
     * This method is used by the 'accessControl' filter.
```

```
     * @return array access control rules
```

```
     */
```

```
    public function accessRules()
```

```
    {
```

```
return array(
    array('allow', // allow all users to perform 'index' and 'view' actions
        'actions'=>array('index','view','saludo','antidoping','Inasistente',
                        'apto','noApto','resultados','ascenso'),
        'roles'=>array('admin','demo','Evaluador'),
    ),
    array('allow', // allow authenticated user to perform 'create' and 'update' actions
        'actions'=>array('create','update'),
        'roles'=>array('admin'),
    ),
    array('allow', // allow admin user to perform 'admin' and 'delete' actions
        'actions'=>array('admin','delete','saludo','Reporte',
                        'Reportes','CargarDatosReporte','Postulacion',
                        'desempenho','cargarModelo','suma',
                        'Examen','Evaluador','VistaCompleta',
                        'desactivarVistaCompleta'),
        'roles'=>array('admin'),
    ),
    array('deny', // deny all users
        'users'=>array('*'),
    ),
);
}
```

```
/**
 * Muestra una vista preparada donde recibe como parametro.
 * @param integer $id el ID del modelo que se cargara
 * llama a difentes funciones para setear las variables de envio
 */
public function actionView($id)
{
    $this->render('view',array(
        'model'=>$this->loadModel($id),
        'data1' => $this->actionVistaEvaluador($id), #FuncRecepcion
        'desempeno' => $this->actionDesempenho($id), #desempeño
        'data2' => $this->actionExamen($id) , #resultados
    ));
}

public function actionResultados()
{
    $model=Funcionarios::model()->cargarCedula(Yii::app()->user->getState('cedula'));
    $this->render('view',array(
        'model'=>$this->loadModel($model->id_funcionario),
        'data1' => $this->actionVistaEvaluador($model->id_funcionario), #FuncRecepcion
        'desempeno' => $this->actionDesempenho($model->id_funcionario), #desempeño
        'data2' => $this->actionExamen($model->id_funcionario) , #resultados
    ));
}
```

```
public function actionAscenso()
{
    $model=Funcionarios::model()->cargarCedula(Yii::app()->user->getState('cedula'));
    $this->render('ascenso',array(
        'model'=>$this->loadModel($model->id_funcionario,));
}

/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function actionCreate()
{
    $model=new Funcionarios;

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Funcionarios']))
    {
        $model->attributes=$_POST['Funcionarios'];
        if($model->save()){

            #asignar antigüedad
            $antigüedad = new Antigüedad;
            $antigüedad->guardarAntigüedad($_POST['Funcionarios']['Cedula'],
```



```
$_POST['Funcionarios']['Fecha_Ingreso']);
```

```
#asignar nivel academico
```

```
$nivel = new FuncionarioNivelAcademico;
```

```
$nivel->guardarNivelAcademico($_POST['Funcionarios']['Cedula'],
```

```
$_POST['Funcionarios']['des_nivel_acade']);
```

```
Yii::app()->user->setFlash('success', "Ha sido registrado exitosamente");
```

```
$this->redirect(array('view','id'=>$model->id_funcionario));}
```

```
}
```

```
$this->render('create',array(
```

```
'model'=>$model,
```

```
));
```

```
}
```

```
/**
```

```
 * Updates a particular model.
```

```
 * If update is successful, the browser will be redirected to the 'view' page.
```

```
 * @param integer $id the ID of the model to be updated
```

```
 */
```

```
public function actionUpdate($id)
```

```
{
```

```
    $model=$this->loadModel($id);
```

```
    $nivel = new FuncionarioNivelAcademico;
```

```
    $nivel1 = new FuncionarioNivelAcademico;
```

```
$nivel1=$nivel->cargarModelo($model->Cedula);

// Uncomment the following line if AJAX validation is needed
// $this->performAjaxValidation($model);

if(isset($_POST['Funcionarios']))
{
    $model->attributes=$_POST['Funcionarios'];
    $nivel->actualizarNivelAcademico($_POST['Funcionarios']['Cedula'],

$_POST['Funcionarios']['des_nivel_acade']);

    if($model->save()){
        //msj
        Yii::app()->user->setFlash('success', "Sus datos han sido actualizados");

        $this->redirect(array('view','id'=>$model->id_funcionario));
    }

    $this->render('update',array(
        'model'=>$model,
        'nivel'=>$nivel1,
    ));
}

/**
 * Deletes a particular model.
 *
 * If deletion is successful, the browser will be redirected to the 'admin' page.
```

```
* @param integer $id the ID of the model to be deleted
*/

public function actionDelete($id)
{
    $this->loadModel($id)->delete();

    Yii::app()->user->setFlash('success', "Eliminado satisfactoriamente");

    // if AJAX request (triggered by deletion via admin grid view), we should not redirect the
browser
    if(!isset($_GET['ajax']))
        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));

}

/**
 * Lists all models.
 */
public function actionIndex()
{
    # inicializo

    $dataProvider=new CActiveDataProvider('Funcionarios');

    if (isset($_GET['pdf'])){
        $mPDF1 = Yii::app()->ePdf->mpdf();

        $mPDF1->WriteHTML($this->renderPartial('index',
array('dataProvider'=>$dataProvider), true));

        //hacemos un render parcial a una vista preparada, en este caso es la
vista pdfRepor
```

```
        $mPDF1->Output();

    }

    $this->render('index',array(

        'dataProvider'=>$dataProvider,

    ));

}

public function actionReportes()

{

    $menu=new menuJerarquiaForm;

    if (isset($_POST["menuJerarquiaForm"]))

    {

        $menu->attributes=$_POST["menuJerarquiaForm"];

        if($menu->validate())

        {

            $model=Funcionarios::model()->findAll('Cod_Jerarquia=:post_id ',array(

                ':post_id'=>$_POST["menuJerarquiaForm"]['codigo'],

                ));# -->devuelve un array de Personas.

            (El modelo sobre el que se realiza la búsqueda)

            $this->actionReporte('reportepdf',$model);

        }

    }

    $this->render('reporte',array(

        'model'=>$menu,

    ));

}
```

```
public function actionCargarDatosReporte($id)
{
    $model=$this->loadModel($id);# -->devuelve un array de Personas. (El modelo sobre el que se
realiza la búsqueda)

    $this->actionReporte('reportepdf',$model);
}

public function actionReporte($vista,$data)
{
    ini_set('max_execution_time', 300);#aumento el tiempo de esperar en generar mi
pdf de esta manera no ocasiona problemas equivale a max_execution_time=30 en php.ini

    $mPDF1 = Yii::app()->ePdf->mpdf();

    #$header='';
    $header='

<table width="100%"><tr>
<td width="60%" style="color:black;"><span style="font-weight: bold; font-size: 14pt;">
Bomberos del Estado Miranda</span><br />Sistema de Ascenso de los Bomberos del Estado Miranda<br
/><span style="font-size: 15pt;">
</span>RIF:09432477230947jdas / TELF:0212-232355</td>
<td width="40%" style="text-align: right;"></td>
</tr></table>

';

    $mPDF1=new mPDF('win-1252','LETTER','',15,15,25,12,5,7);

    //hacemos un render partial a una vista preparada, en este caso es la
vista pdfRepor

    $mPDF1->SetHTMLHeader($header);
```

```
$mPDF1->SetFooter(' {DATE j/m/Y}| Página {PAGENO}| Reporte de  
Funcionarios');  
  
$mPDF1->WriteHTML($this->renderPartial($vista, array('model'=>$data), true));  
  
$mPDF1->Output('Reporte','D');  
  
}
```

```
/**
```

```
 * Manages all models.
```

```
 */
```

```
public function actionAdmin()
```

```
{
```

```
    $model=new Funcionarios('search');
```

```
    $model->unsetAttributes(); // clear any default values
```

```
    if(isset($_GET['Funcionarios']))
```

```
        $model->attributes=$_GET['Funcionarios'];
```

```
    $this->render('admin',array(
```

```
        'model'=>$model,
```

```
    ));
```

```
}
```

```
/**
```

```
 * Returns the data model based on the primary key given in the GET variable.
```

```
 * If the data model is not found, an HTTP exception will be raised.
```

```
 * @param integer $id the ID of the model to be loaded
```

```
 * @return Funcionarios the loaded model
```

```
* @throws CHttpException
*/

public function loadModel($id)
{
    $model=Funcionarios::model()->findByPk($id);
    if($model===null)
        throw new CHttpException(404,'The requested page does not exist.');
```

```
/**
 * Performs the AJAX validation.
 * @param Funcionarios $model the model to be validated
 */
protected function performAjaxValidation($model)
{
    if(isset($_POST['ajax']) && $_POST['ajax']=='funcionarios-form')
    {
        echo CActiveForm::validate($model);
        Yii::app()->end();
    }
}
```

```
public function actionSaludo()
{
```

```
$this->layout='//layouts/column2';

    $this->render('saludo',

        array('model'=>$model=Funcionarios::model()->cargarCedula(yii::app()->user-
>getState('cedula'))));
    }

#cargo el modelo

#verifico si esta activo

#verifico años de antigüedad

#verificar último año de postulación

public function actionPostulacion($id)
{
    $model=$this->loadModel($id);

    $msg="";

    if ($model->Cod_Condicion === '2') #2 es que esta en condición de inactivo
    {

        $msg ='actualmente no se encuentra activo lo sentimos no puede proceder a postularse ';

    }

    else
    {

        $anos=new Antigüedad;

        $antigüedad=new Antigüedad;

        $antigüedad=$anos->cargarCedula($model->Cedula);

        if ($antigüedad === null)
```



```
{  
    $msg.=' No pudimos encontrar la antigüedad asociada a tu identificación  
,actualmente no tienes el tiempo suficiente para optar por un ascenso';  
  
}  
else  
{  
    $jerarquiaAscender=$model->verificarJerarquia();#jerarquia inmediata a  
ascender  
    if ($model->cumpleAnhosAntigüedad($jerarquiaAscender,$antigüedad-  
>num_anos))  
        $msg.=' Cumple con los requisitos mínimos de tiempo  
para postularse ';  
    else  
        $msg.=' No Cumple con los requisitos mínimos de  
tiempo para postularse ';  
    }  
  
}  
  
Yii::app()->user->setFlash('notice',$msg );  
$this->actionView($id);  
}  
  
public function suma($a=null,$b=null,$c=null,$d=null,$e=null)  
{  
    return $suma = $a + $b + $c + $d + $e;  
  
    # return $suma = $espíritu->num_TotEspNota + $competencia->num_TotComNota + $desempenho-  
>num_TotDesemNota + $aptitud->num_TotAcNota + $habilidad->num_TotHaNota;
```

```
}

/*Instanciar espiritu,competencia,desempeño,aptitud y habilidad
* tratamientos y sumatorias de notas
* finalmente devuelve una nota final y una vista renderizada con todas las notas
*/

public function actionDesempenho($id)
{
    $model=$this->loadModel($id);

    $cedula=$model->Cedula;

    $espiritu = EspirituBomberil::model()->CargarModelo($cedula)? (double)EspirituBomberil::model()-
    >CargarModelo($cedula)->num_TotEspNota :0;

    $competencia= Competencia::model()->CargarModelo($cedula) ? (double)Competencia::model()-
    >CargarModelo($cedula)->num_TotComNota: 0;

    $desempenho= Desempenho::model()->CargarModelo($cedula) ? (double)Desempenho::model()-
    >CargarModelo($cedula)->num_TotDesemNota : 0;

    $aptitud= AptitudActitud::model()->CargarModelo($cedula) ? (double)AptitudActitud::model()-
    >CargarModelo($cedula)->num_TotAcNota : 0;

    $habilidad= Habilidad::model()->CargarModelo($cedula)? (double)Habilidad::model()-
    >CargarModelo($cedula)->num_TotHaNota :0;

    $arrayD = array('EspirituBomberil' => $espiritu,
                    'competencia'=> $competencia,
                    'desempenho' => $desempenho,
                    'aptitud' => $aptitud,
                    'habilidad' => $habilidad,
```

```
'suma' => $this->suma($espíritu,$competencia,$desempenho,$aptitud,$habilidad) );

return $arrayD;
}

/*Instanciar
* tratamientos y sumatorias de notas
* finalmente devuelve una nota final y una vista renderizada con todas las notas
*/

public function actionExamen($id)
{
    $exámenes = array(0 => false,1 => false,2 => false,3 => "",4 => "",5 => "", );

    $model=$this->loadModel($id);
    $cedula=$model->Cedula;
    $msg="";
    $fisico= Fisica::model()->CargarModelo($cedula);
    $medico= ExamenMedico::model()->CargarModelo($cedula);
    $antidoping= Antidoping::model()->CargarModelo($cedula);
    if($model->Cod_Condicion != 2)
    {
        if (($fisico === false) or ($medico === false) or ($antidoping === false) )
        {
            $msg=' No hemos podido cargar los exámenes medicos, revise si estan cargadas ';
            $msg.="";
        }
    }
    else
    {
        if ($fisico->des_fisica_condicion == 'APTO') {
```

```
$exámenes[3] ='El resultado de Exámenes : Fisico '.$fisico->des_fisica_condicion;

                $exámenes[0] = true;

        }

else{

        $exámenes[3] ='El resultado de Exámenes : Fisico '.$fisico->des_fisica_condicion;

$exámenes[0] = false;

}

if ($medico->des_asist_exa_condicion == 'APTO'){

        $exámenes[4] =' Medico '.$medico->des_asist_exa_condicion;

        $exámenes[1] = true;

                }

else{

        $exámenes[4] =' Medico '.$medico->des_asist_exa_condicion;

$exámenes[1] = false;

}

if ($antidoping->des_antid_condicion === null or $antidoping->des_antid_condicion === "" ){

        $exámenes[5] =' Antidoping Inasistente';

                $exámenes[2] = false;

}

else{

        $exámenes[5] =' Antidoping '.$antidoping->des_antid_condicion;

                $exámenes[2] = true;

}

$msg .='<br> <a href="'.$yii::app()->createUrl('/notas/create', array('ced'=>'1')).'"> Ver Detalles

</a>';

}
```

```
}
```

```
# Yii::app()->user->setFlash('notice',$msg );
```

```
# $this->render('view',array(
```

```
    #'model'=>$model,
```

```
    # ));
```

```
    return $exámenes;
```

```
}
```

```
public function actionVistaEvaluador($id){
```

```
    $model=$this->loadModel($id);
```

```
    $cedula=$model->Cedula;
```

```
    $buscarEval= Postulados::model()->getFuncionarioRecepcion($cedula);
```

```
    if ($buscarEval){
```

```
        $evaluador = Funcionarios::model()->CargarModelo($buscarEval);
```

```
        return $evaluador;
```

```
    }
```

```
    else return null;
```

```
}
```

```
public function actionAntidoping(){
```

```
    echo "Antidoping negativo";
```

```
}
```

```
public function actionApto(){
```

```
    echo "Apto ";
```

```
}
```

```
public function actionNoApto(){
```

```
    echo "No Apto";
```

```
}
```

```
public function actionInasistente(){
```

```
    echo "Antidoping Inasistente";
```

```
}
```

```
}
```

Controlador Notificación:

```
<?php
```

```
class NotificacionController extends Controller
```

```
{
```

```
    /**
```

```
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning
```

```
     * using two-column layout. See 'protected/views/layouts/column2.php'.
```

```
     */
```

```
public $layout='//layouts/column2';
```

```
/**
```

```
 * @return array action filters
```

```
 */
```

```
public function filters()
```

```
{
```

```
    return array(
```

```
        'accessControl', // perform access control for CRUD operations
```

```
        'postOnly + delete', // we only allow deletion via POST request
```

```
    );
```

```
}
```

```
/**
```

```
 * Specifies the access control rules.
```

```
 * This method is used by the 'accessControl' filter.
```

```
 * @return array access control rules
```

```
 */
```

```
public function accessRules()
```

```
{
```

```
    return array(
```

```
        array('allow', // allow all users to perform 'index' and 'view' actions
```

```
            'actions'=>array('index','view'),
```

```
            'users'=>array('*'),
```

```
        ),
```

```
        array('allow', // allow authenticated user to perform 'create' and 'update' actions
```

```
            'actions'=>array('create','update'),
```

```
            'users'=>array('@'),
```

```
        ),
```

```
        array('allow', // allow admin user to perform 'admin' and 'delete' actions
            'actions'=>array('admin','delete'),
            'roles'=>array('admin'),
        ),
        array('deny', // deny all users
            'users'=>array('*'),
        ),
    );
}

/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $this->render('view',array(
        'model'=>$this->loadModel($id),
    ));
}

/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function actionCreate()
{
    $model=new Notificacion;
```

```
// Uncomment the following line if AJAX validation is needed

// $this->performAjaxValidation($model);

if(isset($_GET['ced']))

{

$model->cedula_enviado=$_GET['ced_send'];

# usuario::model()->find(where cedula=?)

#     $model->mensaje='notificacion personalizada';

}

$model->cedula=yii::app()->user->getState('cedula');

if(isset($_POST['Notificacion']))

{

    $model->attributes=$_POST['Notificacion'];

    if($model->save()){

        $correo= new enviarEmail();

        if($correo->enviarCorreo(

            $model->attributes['email'],

            $model->attributes['asunto'],

            $model->attributes['mensaje']))

        {

            $msg='Mensaje enviado correctamente';

            Yii::app()->user->setFlash('success',$msg );

        }

        else

        {

            $msg='Se presento un problema en el envio';

            Yii::app()->user->setFlash('error',$msg );

        }

    }

}
```

```
        $this->redirect(array('view','id'=>$model->id));}

    }

    $this->render('create',array(

        'model'=>$model,

    ));

}

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */
public function actionUpdate($id)
{
    $model=$this->loadModel($id);

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Notificacion']))
    {
        $model->attributes=$_POST['Notificacion'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id));
    }
}
```

```
$this->render('update',array(
    'model'=>$model,
));
}

/**
 * Deletes a particular model.
 * If deletion is successful, the browser will be redirected to the 'admin' page.
 * @param integer $id the ID of the model to be deleted
 */
public function actionDelete($id)
{
    $this->loadModel($id)->delete();

    // if AJAX request (triggered by deletion via admin grid view), we should not redirect the
browser
    if(!isset($_GET['ajax']))
        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
}

/**
 * Lists all models.
 */
public function actionIndex()
{
    $dataProvider=new CActiveDataProvider('Notificacion');
    $this->render('index',array(
        'dataProvider'=>$dataProvider,
    ));
}
```

```
}

/**
 * Manages all models.
 */
public function actionAdmin()
{
    $model=new Notificacion('search');
    $model->unsetAttributes(); // clear any default values
    if(isset($_GET['Notificacion']))
        $model->attributes=$_GET['Notificacion'];

    $this->render('admin',array(
        'model'=>$model,
    ));
}

/**
 * Returns the data model based on the primary key given in the GET variable.
 * If the data model is not found, an HTTP exception will be raised.
 * @param integer $id the ID of the model to be loaded
 * @return Notificacion the loaded model
 * @throws CHttpException
 */
public function loadModel($id)
{
    $model=Notificacion::model()->findByPk($id);
    if($model===null)
        throw new CHttpException(404,'The requested page does not exist.');
```

```
        return $model;
    }

    /**
     * Performs the AJAX validation.
     * @param Notificacion $model the model to be validated
     */
    protected function performAjaxValidation($model)
    {
        if(isset($_POST['ajax']) && $_POST['ajax']=='notificacion-form')
        {
            echo CActiveForm::validate($model);
            Yii::app()->end();
        }
    }
}
```

Controlador Alerta

<?php

```
class AlertaController extends Controller
{
    /**
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning
     * using two-column layout. See 'protected/views/layouts/column2.php'.
     */
    public $layout='//layouts/column2';

    /**
     * @return array action filters
     */
}
```

```
*/  
  
public function filters()  
{  
    return array(  
        'accessControl', // perform access control for CRUD operations  
        'postOnly + delete', // we only allow deletion via POST request  
    );  
}  
  
/**  
 * Specifies the access control rules.  
 * This method is used by the 'accessControl' filter.  
 * @return array access control rules  
 */  
  
public function accessRules()  
{  
    return array(  
        array('allow', // allow all users to perform 'index' and 'view' actions  
            'actions'=>array('index'),  
            'users'=>array('@'),  
        ),  
        array('allow', // allow admin user to perform 'admin' and 'delete' actions  
            'actions'=>array('admin','delete'),  
            'roles'=>array('admin','demo','Evaluador'),  
        ),  
        array('deny', // deny all users  
            'users'=>array('*'),  
        ),  
    );  
}
```

```
}

/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $this->render('view',array(
        'model'=>$this->loadModel($id),
    ));
}

/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function createAction()
{
    $model=new Alerta;

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Alerta']))
    {
        $model->attributes=$_POST['Alerta'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id));
    }
}
```

```
}

$this->render('create',array(

    'model'=>$model,

));

}

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */
public function actionUpdate($id)
{
    $model=$this->loadModel($id);

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Alerta']))
    {
        $model->attributes=$_POST['Alerta'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id));
    }

    $this->render('update',array(

        'model'=>$model,

    ));
}
```

```
}

/**
 * Deletes a particular model.
 * If deletion is successful, the browser will be redirected to the 'admin' page.
 * @param integer $id the ID of the model to be deleted
 */
public function actionDelete($id)
{
    $this->loadModel($id)->delete();

    // if AJAX request (triggered by deletion via admin grid view), we should not redirect the
browser
    #if(!isset($_GET['ajax']))
    #        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
    $this->redirect(array('index'));
}

/**
 * Lists all models.
 */
public function actionIndex()
{
    $criteria=new CDbCriteria(array(
        'order'=>'id desc',
        'condition'=>'cedula_funcionario='.yii::app()->user->getState('cedula'),

    ));

    $dataProvider=new CActiveDataProvider('Alerta', array(
```

```
'criteria'=>$criteria,));
```

```
        $dataProvider=new CActiveDataProvider('Alerta');  
        $this->render('index',array(  
            'dataProvider'=>$dataProvider,  
        ));  
    }
```

```
/**
```

```
 * Manages all models.
```

```
 */
```

```
public function actionAdmin()
```

```
{
```

```
    $model=new Alerta('search');
```

```
    $model->unsetAttributes(); // clear any default values
```

```
    if(isset($_GET['Alerta']))
```

```
        $model->attributes=$_GET['Alerta'];
```

```
    $this->render('admin',array(  
        'model'=>$model,  
    ));
```

```
}
```

```
/**
```

```
 * Returns the data model based on the primary key given in the GET variable.
```

```
 * If the data model is not found, an HTTP exception will be raised.
```

```
 * @param integer $id the ID of the model to be loaded
```

```
 * @return Alerta the loaded model
```

```
* @throws CHttpException
*/

public function loadModel($id)
{
    $model=Alerta::model()->findByPk($id);
    if($model===null)
        throw new CHttpException(404,'The requested page does not exist.');
```



```
return $model;
}

/**
 * Performs the AJAX validation.
 * @param Alerta $model the model to be validated
 */
protected function performAjaxValidation($model)
{
    if(isset($_POST['ajax']) && $_POST['ajax']=='alerta-form')
    {
        echo CActiveForm::validate($model);
        Yii::app()->end();
    }
}

public function actionGenerarAlerta()
{
    $model=new Alerta('search');
    $model->unsetAttributes(); // clear any default values
    if(isset($_GET['Alerta']))
        $model->attributes=$_GET['Alerta'];
```

```
$this->render('admin',array(
    'model'=>$model,
));
}

}
```

CLASE ASCENSO

```
class Ascenso extends CActiveRecord{

    // ----- CALCULO DE PUNTAJE CON AÑOS DE SERVICIO -----

    public static function getAnosServicio($funcionario){
        $anos_antiguedad = new Antiguedad;
        $criteria = new CDbCriteria(
            array(
                'select'=>'Fecha_Ingreso',
                'condition'=>'Cedula=:ci',
                'params'=>array(
                    ':ci'=>$funcionario
                )
            )
        );

        $fecha_ingreso_func = Funcionarios::model()->find($criteria);
        return $anos_antiguedad->calcularAnos($fecha_ingreso_func->Fecha_Ingreso); //Retorna
        la cantidad de años de servicio

    }
```

```
public static function getPuntajeAnosServicio($cant_anos){  
    if ($cant_anos > 20) return 20; //Si es mayor a 20 siempre sera 20 ptos  
    else  
        return $cant_anos; //sino retorna la cantidad de años como puntaje (0-20)  
}  
  
public static function obtenerNotaAnosServicios($funcionario){  
    return Ascenso::getPuntajeAnosServicio(Ascenso::getAnosServicio($funcionario));  
}  
  
// ----- CALCULO DE PUNTAJE NOTAS DESEMPEÑO -----  
  
// ----- ESPIRITU BOMBERIL -----  
  
public static function getnotaEspirituBomberilFinal($funcionario,$conf_asc_fecha){  
    $criteria = new CDbCriteria(  
        array(  
            'select'=>'num_TotEspPorc',  
            'condition'=>'id_conf_asc_fecha=:conf_fecha AND  
Cedula=:ci',  
            'params'=>array(  
                'conf_fecha'=>$conf_asc_fecha,  
                'ci'=>$funcionario  
            )  
        )  
    );  
    if (EspirituBomberil::model()->exists($criteria)){  
        $nota_final = EspirituBomberil::model()->find($criteria);  
        return $nota_final->num_TotEspPorc;  
    }
```

```
    }  
    return 0.000;  
    //return $nota_final;  
  
}  
  
// ----- COMPETENCIA -----  
  
public static function getnotaCompetenciaFinal($funcionario,$conf_asc_fecha){  
    $criteria = new CDbCriteria(  
        array(  
            'select'=>'num_TotComPorc',  
            'condition'=>'id_conf_asc_fecha=:conf_fecha AND  
Cedula=:ci',  
            'params'=>array(  
                ':conf_fecha'=>$conf_asc_fecha,  
                ':ci'=>$funcionario  
            )  
        )  
    );  
    if (Competencia::model()->exists($criteria)){  
        $nota_final = Competencia::model()->find($criteria);  
        return $nota_final->num_TotComPorc;  
    }  
    return 0.000;  
    //return $nota_final;  
  
}
```

// ----- DESEMPEÑO -----

```
public static function getnotaDesempenhoFinal($funcionario,$conf_asc_fecha){  
    $criteria = new CDbCriteria(  
        array(  
            'select'=>'num_TotDesemPorc',  
            'condition'=>'id_conf_asc_fecha=:confg_fecha AND  
Cedula=:ci',  
            'params'=>array(  
                ':confg_fecha'=>$conf_asc_fecha,  
                ':ci'=>$funcionario  
            )  
        )  
    );  
    if (Desempenho::model()->exists($criteria)){  
        $nota_final = Desempenho::model()->find($criteria);  
        return $nota_final->num_TotDesemPorc;  
    }  
    return 0.000;  
    //return $nota_final;  
}
```

// ----- APTITUD-ACTITUD -----

```
public static function getnotaAptitudActitudFinal($funcionario,$conf_asc_fecha){
```

```
$criteria = new CDbCriteria(
    array(
        'select'=>'num_TotApPorc',
        'condition'=>'id_conf_asc_fecha=:conf_fecha AND
Cedula=:ci',
        'params'=>array(
            ':conf_fecha'=>$conf_asc_fecha,
            ':ci'=>$funcionario
        )
    )
);

if (AptitudActitud::model()->exists($criteria)){
    $nota_final = AptitudActitud::model()->find($criteria);
    return $nota_final->num_TotApPorc;
}

return 0.000;

//return $nota_final;

}

// ----- HABILIDAD -----

public static function getnotaHabilidadFinal($funcionario,$conf_asc_fecha){
    $criteria = new CDbCriteria(
        array(
            'select'=>'num_TotHaPorc',
            'condition'=>'id_conf_asc_fecha=:conf_fecha AND
Cedula=:ci',
```

```
'params'=>array(

    ':config_fecha'=>$conf_asc_fecha,

    ':ci'=>$funcionario

)

)

);

if (Habilidad::model()->exists($criteria)){

    $nota_final = Habilidad::model()->find($criteria);

    return $nota_final->num_TotHaPorc;

}

return 0.000;

//return $nota_final;

}

public static function obtenerNotaFinalDesempenho($funcionario,$conf_asc_fecha){

    $acum = 0;

    $acum += Ascenso::getnotaEspirituBomberilFinal($funcionario,$conf_asc_fecha);

    $acum += Ascenso::getnotaCompetenciaFinal($funcionario,$conf_asc_fecha);

    $acum += Ascenso::getnotaDesempenhoFinal($funcionario,$conf_asc_fecha);

    $acum += Ascenso::getnotaAptitudActitudFinal($funcionario,$conf_asc_fecha);

    $acum += Ascenso::getnotaHabilidadFinal($funcionario,$conf_asc_fecha);

    return $acum;

}

/*return Ascenso::getnotaEspirituBomberilFinal($funcionario,$conf_asc_fecha)+

    Ascenso::getnotaCompetenciaFinal($funcionario,$conf_asc_fecha)+
```

```
Ascenso::getnotaDesempenhoFinal($funcionario,$conf_asc_fecha)+
Ascenso::getnotaAptitudActitudFinal($funcionario,$conf_asc_fecha)+
Ascenso::getnotaHabilidadFinal($funcionario,$conf_asc_fecha);*/

}

// ----- CALCULAR EQUIVALENTE A NOTA FINAL -----

public static function obtenerEquivalenteFinal($tipo_evaluacion){
    switch ($tipo_evaluacion){
        case 'academico':
        case 'antiguedad':
        case 'desempenho':
        case 'mejoramiento_profesional': return 0.20;
        case 'cursos': return 0.16;
        case 'condecoraciones': return 0.04;
    }
}

/*
Este metodo se utiliza para calcular la nota final promediada de todos los aspectos
evaluados
(desempeños, condecoraciones, años de servicio, cursos, nivel academico, etc)
*/

public static function obtenerNotaFinalDeAscenso($ci_funcionario,$conf_asc_fecha){
    $acum = 0.000;

    $acum += Ascenso::obtenerNotaAnosServicios($ci_funcionario) *
Ascenso::obtenerEquivalenteFinal('antiguedad');
```

```
$acum += Ascenso::obtenerNotaFinalDesempenho($ci_funcionario,$conf_asc_fecha) *
Ascenso::obtenerEquivalenteFinal('desempenho');

$acum += Ascenso::obtenerNotaFinalNivelAcademico($ci_funcionario) *
Ascenso::obtenerEquivalenteFinal('academico'); //NIVEL ACADEMICO

$acum += 20 * Ascenso::obtenerEquivalenteFinal('mejoramiento_profesional');
//MEJORAMIENTO PROFESIONAL

$acum += 16 * Ascenso::obtenerEquivalenteFinal('cursos'); //DIPLOMADOS Y CURSOS

$acum += 0.16 * Ascenso::obtenerEquivalenteFinal('condecoraciones');
//CONDECORACIONES

return $acum;
}

// ----- CALCULAR NOTA FINAL DEL NIVEL ACADEMICO -----

public static function obtenerNotaFinalNivelAcademico($funcionario){

    $nivel_aca = Funcionarios::getNotaNivelAcademico($funcionario);

    return $nivel_aca;

}

// ----- SABER SI UN FUNCIONARIO ES APTO PARA ASCENDER -----
-----

public static function isApto($ci_funcionario,$conf_asc_fecha){

    return (Ascenso::mediApto($ci_funcionario,$conf_asc_fecha))

        && (Ascenso::fisiApto($ci_funcionario,$conf_asc_fecha)) //Si todo
verdadero, es apto (true)

        && (Ascenso::dopingApto($ci_funcionario,$conf_asc_fecha)) //A la primera
falsa, se corta la búsqueda y no es apto (false)

        && (Ascenso::psiquiApto($ci_funcionario,$conf_asc_fecha));

}
```

```
public static function mediApto($ci_funcionario,$conf_asc_fecha){

    $criteria_medica = new CDbCriteria(
        array(

'select'=>'Cedula,des_asist_exa_medico,des_asist_exa_condicion,id_conf_asc_fecha',
        'condition'=>'id_conf_asc_fecha=:conf_fech AND Cedula=:ci',
        'params'=>array(
                                'conf_fech'=>$conf_asc_fecha,
                                'ci'=>$ci_funcionario
                            )
        )
    );

    if (!ExamenMedico::model()->exists($criteria_medica))
        return false;

    else{//si existe
        $medica = ExamenMedico::model()->find($criteria_medica); //leer el valor de la
bdd

        if ($medica->des_asist_exa_medico == "INASISTENTE")//si inasistente
            return false;

        else{//si asistente
            if ($medica->des_asist_exa_condicion == "NO APTO")
                return false;

            }
        }

    return true;

}
```

```
public static function fisiApto($ci_funcionario,$conf_asc_fecha){

    $criteria_fisica = new CDbCriteria(

        array(

            'select'=>'Cedula,des_fisica_asistencia,des_fisica_condicion,id_conf_asc_fecha',

            'condition'=>'id_conf_asc_fecha=:conf_fech AND Cedula=:ci',

            'params'=>array(

                ':conf_fech'=>$conf_asc_fecha,

                ':ci'=>$ci_funcionario

            )

        )

    );

    //$fisica = Fisica::model()->exists($criteria_fisica);//find($criteria_fisica);

    if (!Fisica::model()->exists($criteria_fisica))

        return false;

    else{//si existe

        $fisica = Fisica::model()->find($criteria_fisica); //leer el valor de la bdd

        if ($fisica->des_fisica_asistencia == "INASISTENTE")//si inasistente

            return false;

        else{//si asistente

            if ($fisica->des_fisica_condicion == "NO APTO")

                return false;

        }

    }

    return true;

}

public static function dopingApto($ci_funcionario,$conf_asc_fecha){
```

```
$criteria_antidoping = new CDbCriteria(
    array(

'select'=>'Cedula,des_antid_asistencia,des_antid_condicion,id_conf_asc_fecha',
        'condition'=>'id_conf_asc_fecha=:conf_fech AND Cedula=:ci',
        'params'=>array(
                                ':conf_fech'=>$conf_asc_fecha,
                                ':ci'=>$ci_funcionario
                            )
            )
    );

//$antidoping = Antidoping::model()->exists($criteria_antidoping);
//find($criteria_antidoping);

if (!Antidoping::model()->exists($criteria_antidoping))
    return false;

else{//si existe

    $antidoping = Antidoping::model()->find($criteria_antidoping); //leer el valor de la
bdd

    if ($antidoping->des_antid_asistencia == "INASISTENTE")//si inasistente
        return false;

    else{//si asistente

        if ($antidoping->des_antid_condicion == "NO APTO" || $antidoping-
>des_antid_condicion == "POSITIVO")

            return false;

        }

    }

    return true;

}
```

```
public static function psiquiApto($ci_funcionario,$conf_asc_fecha){  
    $criteria_psiquica = new CDbCriteria(  
        array(  
  
        'select'=>'Cedula,des_psi_asistencia,des_psi_condicion,id_conf_asc_fecha',  
        'condition'=>'id_conf_asc_fecha=:conf_fech AND Cedula=:ci',  
        'params'=>array(  
            ':conf_fech'=>$conf_asc_fecha,  
            ':ci'=>$ci_funcionario  
        )  
    )  
);  
    //$psiquica = Psiquica::model()->exists($criteria_psiquica);// find($criteria_psiquica);  
    if (!Psiquica::model()->exists($criteria_psiquica))  
        return false;  
    else{//si existe  
        $psiquica = Psiquica::model()->find($criteria_psiquica); //leer el valor de la bdd  
        if ($psiquica->des_psi_asistencia == "INASISTENTE")//si inasistente  
            return false;  
        else{//si asistente  
            if ($psiquica->des_psi_condicion == "NO APTO")  
                return false;  
        }  
    }  
    return true;  
}  
  
// ----- METODOS PARA OBTENER LA NOTA MINIMA FINAL PARA ASCENDER SEGUN LA  
GERARQUIA A LA QUE ASPIRA -----  
  
public static function notaMinimaPorJerarquia($jerarquia){
```

```
switch ($jerarquia){  
    case 'CORONEL':  
    case 'TENIENTE CORONEL':  
    case 'MAYOR':  
    case 'CAPITAN':  
    case 'TENIENTE':  
    case 'SUBTENIENTE': return 14.00;  
  
    case 'SARGENTO AYUDANTE':  
    case 'SARGENTO PRIMERO':  
    case 'SARGENTO SEGUNDO': return 13.00;  
  
    case 'CABO PRIMERO':  
    case 'CABO SEGUNDO':  
    case 'DISTINGUIDO': return 12.00;  
  
    case 'BOMBERO':  
    case 'ALUMNO':  
    case 'CIVIL': return 0.00;  
  
    }  
}  
  
public static function obtenerNombreJerarquia($Cod_Jerarquia){  
    $jerarquia = new Funcionarios;  
    return $jerarquia->getNombreJerarquia($Cod_Jerarquia);  
}  
  
public static function obtenerNotaMinimaPorJerarquia($Cod_Jerarquia){
```

```
        return  
Ascenso::notaMinimaPorJerarquia(Ascenso::obtenerNombreJerarquia($Cod_Jerarquia));  
    }
```

```
public static function obtenerPostulados(){  
    $criteria = new CDbCriteria(  
        array(  
            'select'=>'Cedula,id_conf_asc_fecha,Cod_Jerarquia',  
            'condition'=>'id_conf_asc_fecha=:id',  
            'params'=>array(  
                ':id'=>FechaAsc::obtenerUltimoid(),  
            ),  
            'order'=>'Cod_Jerarquia ASC',  
        )  
    );  
    $postulados=Postulados::model()->findAll($criteria);  
    return $postulados;  
}
```

```
public static function obtenerPostuladosAAscender(){  
    $postulados = array();  
    $aprobados = array();  
  
    $j = 0;  
  
    $postulados = Ascenso::obtenerPostulados(); //Obtengo todos los postulados con la ultima  
configuracion de ascenso
```

```
for ($i=0,$size=count($postulados);$i<$size;$i++){  
    $nota_final = 0;  
  
    if (Ascenso::isApto($postulados[$i]->Cedula,$postulados[$i]-  
>id_conf_asc_fecha)){//Si es apto calcula su nota  
  
        $nota_final = Ascenso::obtenerNotaFinalDeAscenso($postulados[$i]-  
>Cedula,$postulados[$i]->id_conf_asc_fecha);  
  
        if ($nota_final >=  
Ascenso::obtenerNotaMinimaPorJerarquia($postulados[$i]->Cod_Jerarquia)){  
  
            $aprobados[$j]['funcionario'] = $postulados[$i];  
            $aprobados[$j++]['nota'] = $nota_final;  
        }  
    }  
  
    else continue; //Sino se lo salta al siguiente registro  
}  
  
if (empty($aprobados)) //Si ninguno es apto o tiene la nota minima entonces retorna null  
    return null;  
  
else{//Sino termina el proceso de ascenso  
    $jer = $aprobados[0]['funcionario']->Cod_Jerarquia; //Tomo la primera jerarquia  
para separarlo en sub arreglos  
    $ascendidos = array();  
    $j = 0;  
  
    for ($i = 0,$size = count($aprobados); $i < $size; $i++){//Separar las jerarquias en  
un arreglo  
        if ($aprobados[$i]['funcionario']->Cod_Jerarquia == $jer){ //Si la jerarquia  
es la misma (si es el primer o unico registro siempre sera la misma)  
            $jerarquia_separada[$j][] = $aprobados[$i];//Lo agrega en un  
arreglo
```

```
}  
else{//Si no es la misma jerarquia  
    $jerarquia_separada[++$j][] = $aprobados[$i];//Lo coloca en una  
nueva posicion del arreglo y  
    $jer = $aprobados[$i]['funcionario']->Cod_Jerarquia;//se cambia  
la jerarquia actual  
}  
}  
  
for ($i = 0,$size = count($jerarquia_separada); $i < $size; $i++){//Luego  
ordenamos por notas y calculamos solo los que ascendieron dependiendo de su plaza configurada en la BDD  
    $jerarquia_separada[$i] =  
Ascenso::ordenarByNota($jerarquia_separada[$i]);//Se ordena por nota de mayor a menor  
    $jerarquia_ascendida =  
Ascenso::calcularAscendidos($jerarquia_separada[$i]);//Luego se retornan por orden los primeros de  
acuerdo a las plazas disponibles  
    $ascendidos = array_merge($ascendidos,$jerarquia_ascendida);// Y se  
van uniendo todos los ascendidos al arreglo principal que será retornado y mostrado en la vista  
    unset($jerarquia_ascendida);  
}  
// $aprobados_ordenados_por_nota = Ascenso::ordenarByNota($aprobados);  
//return $aprobados_ordenados_por_nota;  
return $ascendidos; //Al final tenemos un arreglo con todos los que ascendieron  
  
}  
}  
  
public static function ordenarByNota($array_a_ordenar){  
    foreach ($array_a_ordenar as $key => $row) {  
        $aux[$key] = $row['nota'];  
        //print_r($aux);  
    }  
}
```

```
        array_multisort($aux, SORT_DESC, $array_a_ordenar);

        return $array_a_ordenar;

    }

    public static function calcularAscendidos($postulados){

        $criteria = new CDbCriteria(

            array(

                'select'=>'num_plazas',

                'condition'=>'id_conf_asc_fecha=:id AND Cod_Jerarquia=:jerarquia',

                'params'=>array(

                    ':id'=>$postulados[0]['funcionario']-

>id_conf_asc_fecha,

                    ':jerarquia'=>$postulados[0]['funcionario']-

>Cod_Jerarquia

                )

            )

        );

        $plazas_disponibles = ConfigPlaza::model()->find($criteria);

        if (count($postulados) > $plazas_disponibles->num_plazas){//Si el nro de plazas es menor
al de los postulados se extraen los primeros

            for ($i=0; $i < $plazas_disponibles->num_plazas;$i++){

                $ascendidos[] = $postulados[$i];

            }

            return $ascendidos;

        }

        else return $postulados;// Sino, es porque hay el mismo numero o un numero menor de
postulados que de plazas asi que se retornan los postulados

    }

}
```

CONTROLADOR ASCENSO

```
class AscensoController extends Controller{
```

```
    /**  
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning  
     * using two-column layout. See 'protected/views/layouts/column2.php'.  
     */
```

```
    public $layout='//layouts/column2';
```

```
    /**  
     * @return array action filters  
     */  
    public function filters()  
    {  
        return array(  
            'accessControl', // perform access control for CRUD operations  
            'postOnly + delete', // we only allow deletion via POST request  
        );  
    }
```

```
    /**  
     * Specifies the access control rules.  
     * This method is used by the 'accessControl' filter.  
     * @return array access control rules  
     */  
    public function accessRules()  
    {
```

```
        return array(  
            'allow', // allow all users to access all actions  
            'deny', // deny all users  
            'allow', // allow all users to access all actions  
            'deny', // deny all users  
        );  
    }
```

```
array('allow', // allow all users to perform 'index' and 'view' actions
      'actions'=>array('index','view'),
      'users'=>array('*'),
    ),
array('allow', // allow authenticated user to perform 'create' and 'update' actions
      'actions'=>array('create','update'),
      'users'=>array('@'),
    ),
array('allow', // allow admin user to perform 'admin' and 'delete' actions
      'actions'=>array('admin','delete'),
      'roles'=>array('admin'),
    ),
array('deny', // deny all users
      'users'=>array('*'),
    ),
);
}
```

```
public function actionObtenerPostulados(){
```

```
    $criteria = new CDbCriteria(
        array(
            'condition'=>'id_conf_asc_fecha=:id',
            'params'=>array(
                ':id'=>FechaAsc::obtenerUltimoid()
            )
        )
    );
```

```
$postulados = Postularse::model()->findAll($criteria);  
var_dump($criteria,$postulados);  
}  
  
public function actionView(){  
  
    ini_set('max_execution_time', 300); //Aumentar el tiempo para la solicitud  
  
    $aprobados = Ascenso::obtenerPostuladosAAscender();  
  
    //if (is_null($aprobados)) $aprobados = "No hay data";  
  
    // OBTENER PRIMERO TODOS LOS POSTULADOS DE LA FECHA ACTUAL DE ASCENSO  
(cedula, nombre, jerarquia)  
  
    // DETERMINAR CUALES DE ELLOS ESTAN APTOS (Ya sea porque le falta una nota, o esta no  
    apto de verdad, será agregado al arreglo de reprobados automaticamente, sino al de aprobados y se les  
    calcula sus notas)  
  
    // CALCULAR LAS NOTAS (faltaria nivel academico, condecoraciones y etc) PARA OBTENER  
    LA FINAL QUE ES LA UNICA QUE SE MOSTRARÁ  
  
    // ORDENAR EL ARREGLO POR NOTAS  
  
    // MOSTRAR POSICION, CEDULA, NOMBRE, APELLIDO Y NOTA  
  
    //$aprobados = "Todos los aprobados <br>";  
    $reprobados = "Todos los reprobados<br>";
```

```
//$aprobados = Ascenso::temporalObtenerPostulado(6081498,2);
```

```
$this->render(
    'view',
    array(
        'aprobados'=>$aprobados,
        'reprobados'=>$reprobados
    )
);
}
```

```
public function obtenerPostulados(){
```

```
    $criteria = new CDbCriteria(
        array(
            'select'=>'Cedula,id_conf_asc_fecha',
            'condition'=>'id_conf_asc_fecha=:id',
            'params'=>array(
                ':id'=>2//FechaAsc::obtenerUltimold()
            )
        )
    );

    $postulados=Postulados::model()->findAll($criteria);
    var_dump($postulados);
```

```
}
```

```
public function actionCreate(){
```



```
ini_set('max_execution_time', 300); //Aumentar el tiempo para la solicitud

//$var="asdgljhyasdjkhgjhasdh";

//var_dump($var);

/*$criteria = new CDbCriteria(
    array(
        'select'=>'Cedula,id_conf_asc_fecha',
        'condition'=>'id_conf_asc_fecha=:id',
        'params'=>array(
            ':id'=>FechaAsc::obtenerUltimoid()
        )
    )
);

$postulados=Postulados::model()->findAll($criteria);

foreach($postulados as $row):
    $apto = AscensoController::isApto($row->Cedula,$row->id_conf_asc_fecha);
    var_dump($row,$apto);
endforeach;
*/

echo Ascenso::obtenerNotaAnosServicios(6081498) . " puntos por años de servicio.<br>";
//Prueba de nota por años de servicios

echo Ascenso::getnotaEspirituBomberilFinal(6081498,2) . " puntos (porcentuado) de
espíritu bomberil.<br>"; //Prueba de nota por espíritu bomberil

echo Ascenso::getnotaCompetenciaFinal(6081498,2) . " puntos (porcentuado) de
competencia.<br>"; //Prueba de nota por competencia

echo Ascenso::getnotaDesempenhoFinal(6081498,2) . " puntos (porcentuado) de
desempeño.<br>"; //Prueba de nota por desempeño

echo Ascenso::getnotaAptitudActitudFinal(6081498,2) . " puntos (porcentuado) de aptitud-
actitud.<br>"; //Prueba de nota por actitud_aptitud
```

```
        echo Ascenso::getnotaHabilidadFinal(6081498,2) . " puntos (porcentuado) de
habilidad.<br>"; //Prueba de nota por habilidad

        var_dump(Ascenso::mediApto(6081498,2));

        //echo Ascenso::mediApto(6081498,1)." Resultado de MEDIAPTO<br>"; //PRUEBA
        echo "-----<br>";

        echo "El promedio es: ". Ascenso::obtenerNotaFinalDesempenho(6081498,2) . "<br><br>";

        echo "Nota final de ascenso: ". Ascenso::obtenerNotaFinalDeAscenso(6081498,2);

        // $aprobados = Ascenso::temporalObtenerPostulado(6081498,2);

        //var_dump($aprobados);

        // $nota = Ascenso::getnotaEspirituBomberil(13727546,1);
        // $nota.= " puntos (porcentaje).";
        //var_dump($nota);

    }
}
```

Controlador Postularse

<?php

```
class PostularseController extends Controller
```

```
{
```

```
    /**
```

```
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning
```

```
     * using two-column layout. See 'protected/views/layouts/column2.php'.
```

```
     */
```

```
    public $layout='//layouts/column2';
```

```
/**
 * @return arreglo de actions filters
 */
public function filters()
{
    return array(
        'accessControl', // Ejecuta el control de acceso para el CRUD de operaciones
        'postOnly + delete', // Permitir borrar solo vía POST
    );
}

/**
 * Especifica las reglas de control de acceso
 * Este método es usado por el filtro de control de acceso.
 * @return arreglo de reglas de control de acceso
 */
public function accessRules()
{
    return array(
        array('allow', // Permiso de todos los usuarios.
            'actions'=>array('index','view'),
            'users'=>array('*'),
        ),
        array('allow', // Permisos para los usuarios autenticados.
            'actions'=>array('create'),
            'users'=>array('@'),
        ),
        array('allow', // Permisos para administrador.
```

```
'actions'=>array('admin','delete','rechazar'),
'roles'=>array('super','admin'),
),
array('deny', // Restricciones a los usuarios.
'users'=>array('*'),
),
);
}

/**
 * Muestra un modelo particular.
 * @param integer $id El id del modelo particular
 */
public function actionView($id)
{
    $model = $this->loadModel($id);

    if($model->id_status == 1)
    {
        Yii::app()->user->setFlash('success','Postulación aceptada' );
    }

    if($model->id_status == 2)
    {
        Yii::app()->user->setFlash('error','Postulación rechazada' );
    }

    if($model->id_status == 3)
    {
        Yii::app()->user->setFlash('notice','Postulación en revisión' );
    }
}
```

```
$this->render('view',array(
    'model'=>$model,
));
}

/**
 * Crea un nuevo modelo.
 * Si la creación es exitosa, el navegador redireccionará a la página view.
 */
public function actionCreate()
{
    $model=new Postularse;
    $Fecha_Asc = $model->getFechaAsc();
    $PostExist = $model->getFuncionarioPost();

    if(isset($_POST['Postularse']))
    {
        $cedula=$_POST['Postularse']['cedula'];

        $Funcionario = $model->getFuncionarioCed($cedula);
        $Antiguedad = $model->getUltimaFecha($cedula);
        $MaximoRango = $model->getMaximoRango($Funcionario);

        if($cedula == yii::app()->user->getState('cedula'))
        {
            if($Funcionario->Cod_Condicion == 1)
            {
                if($Funcionario->Cod_Jerarquia < $MaximoRango)
```

```
{  
    if($Antiguedad <= $this->AscensoYear($model-  
>anhosRequeridoJerarquia($Funcionario->Cod_Jerarquia-1)))  
    {  
        if(!is_null($Funcionario))  
        {  
            $model->cedula = $Funcionario-  
>Cedula;  
            $model->nombre = $Funcionario-  
>Nombre;  
            $model->apellido = $Funcionario-  
>Apellidos;  
            $model->cargo = $Funcionario-  
>Cod_Jerarquia;  
            $model->id_status = 3;  
            $model->id_conf_asc_fecha = $model-  
>getFechaAsc()->id_conf_asc_fecha;  
        }  
        if($model->save())  
        {  
            $this->  
>redirect(array('view','id'=>$model->id_postularse));  
        }  
        elseif($Funcionario->Cod_Jerarquia == $MaximoRango)  
        {  
            Yii::app()->user->setFlash('error','Ud. no  
cumple con la antigüedad requerida para postularse' );  
        }  
        else  
        {  
            Yii::app()->user->setFlash('notice','Ud. ya ha alcanzado  
el máximo rango permitido de jerarquía' );  
        }  
        elseif($Funcionario->Cod_Condicion == 2)  
        {  
            Yii::app()->user->setFlash('error','Ud. se encuentra inactivo.' );  
        }  
    }  
}
```

```
    }  
    else  
        Yii::app()->user->setFlash('error','Cédula no es valida. Ingrese su cédula  
para postularse. ');  
    }  
  
    $this->render('create',array(  
        'model'=>$model, 'Fecha_Asc'=>$Fecha_Asc, 'PostExist'=>$PostExist,  
    ));  
}  
  
/**  
 * Rechaza postulación.  
 * Asigna 2 al status del postulante y redirecciona a la página view.  
 */  
public function actionRechazar($cedula)  
{  
    $model = new Postularse;  
    $Postulante = $model->getPostulante($cedula);  
  
    $Postulante->id_status = 2;  
    $Postulante->save();  
    $this->redirect(array('view','id'=>$Postulante->id_postularse));  
}  
  
/**  
 * @return la fecha de antigüedad correspondiente.  
 * @param integer $num el tipo de categoría a calcular.  
 */
```

```
protected function AscensoYear($num)
{
    if($num == 3)
        return date('Y-m-d', strtotime('-3 year'));
    if($num == 2)
        return date('Y-m-d', strtotime('-2 year'));
    if($num == 1)
        return date('Y-m-d', strtotime('-1 year'));
}

/**
 * Actualiza un modelo en particular.
 * Si actualización es exitosa, el navegador redireccionará a página view.
 * @param integer $id el id del modelo a actualizar
 */
public function actionUpdate($id)
{
    $model=$this->loadModel($id);

    if(isset($_POST['Postularse']))
    {
        $model->attributes=$_POST['Postularse'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id_postularse));
    }

    $this->render('update',array(
        'model'=>$model,
    ));
}
```

```
}
```

```
/**
```

```
 * Elimina un modelo en particular.
```

```
 * Si elimina exitosamente, el administrador redireccionará a la página admin.
```

```
 * @param integer $id el id del modelo a eliminar.
```

```
 */
```

```
public function actionDelete($id)
```

```
{
```

```
    $this->loadModel($id)->delete();
```

```
    if(!isset($_GET['ajax']))
```

```
        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
```

```
}
```

```
/**
```

```
 * Lista todos los modelos según la fecha de postulación.
```

```
 */
```

```
public function actionIndex()
```

```
{
```

```
    $model = new Postularse;
```

```
    $criteria = new CDbCriteria;
```

```
    $criteria->addCondition('id_status=2','OR');
```

```
    $criteria->addCondition('id_status=3','OR');
```

```
    $criteria->addCondition('id_conf_asc_fecha=:fecha');
```

```
    if($model->getFechaAsc())
```

```
        $criteria->params=array(':fecha'=>$model->getFechaAsc()->id_conf_asc_fecha);
```

```
    else
```

```
        $criteria->params=array(':fecha'=>0);
```

```
$dataProvider=new CActiveDataProvider('Postularse', array(
    'criteria'=>$criteria,
));

$this->render('index',array(
    'dataProvider'=>$dataProvider,
));
}

/**
 * Gestiona todos los modelos.
 */
public function actionAdmin()
{
    $model=new Postularse('search');

    $model->unsetAttributes(); // Limpia cada valor por defecto.
    if(isset($_GET['Postularse']))
        $model->attributes=$_GET['Postularse'];

    $this->render('admin',array(
        'model'=>$model,
    ));
}

/**
 * Retorna el modelo del basado en la llave primaria dada en el variable GET.
 * Si el modelo no funciona, una excepción HTTP será mostrada.
 * @param integer $id el id del modelo cargado.
 * @return Postularse el modelom cargado
```

```
* @throws CHttpException
*/

public function loadModel($id)
{
    $model=Postularse::model()->findByPk($id);
    if($model===null)
        throw new CHttpException(404,'The requested page does not exist.');
```



```
return $model;
}

/**
 * Ejecua las validaciones AJAX
 * @param Postularse $model el modelo a ser validado.
 */
protected function performAjaxValidation($model)
{
    if(isset($_POST['ajax']) && $_POST['ajax']=='postularse-form')
    {
        echo CActiveForm::validate($model);
        Yii::app()->end();
    }
}
```

Controlador Postulados

```
<?php
```

```
class PostuladosController extends Controller
```

```
{
```

```
    /**
```

```
* @var string the default layout for the views. Defaults to '//layouts/column2', meaning
* using two-column layout. See 'protected/views/layouts/column2.php'.
*/
public $layout='//layouts/column2';

/**
 * @return array action filters
 */
public function filters()
{
    return array(
        'accessControl', // perform access control for CRUD operations
        'postOnly + delete', // we only allow deletion via POST request
    );
}

/**
 * Specifies the access control rules.
 * This method is used by the 'accessControl' filter.
 * @return array access control rules
 */
public function accessRules()
{
    return array(
        array('allow', // allow all users to perform 'index' and 'view' actions
            'actions'=>array('index','view'),
            'users'=>array('*'),
        ),
        array('allow', // allow authenticated user to perform 'create' and 'update' actions
```

```
'actions'=>array('create','update','postulados'),
'users'=>array('@'),
),
array('allow', // allow admin user to perform 'admin' and 'delete' actions
'actions'=>array('admin','delete'),
'roles'=>array('admin'),
),
array('deny', // deny all users
'users'=>array('*'),
),
);
}

/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $model = $this->loadModel($id);

    $criteria = new CDbCriteria;
    $criteria->addCondition('Cedula=:ced');
    $criteria->addCondition('id_conf_asc_fecha=:asc_fecha');
    $criteria->params=array(':ced'=>$model->Cedula,':asc_fecha'=>$model->id_conf_asc_fecha);
    $Espiritu_bomb = EspirituBomberil::model()->find($criteria);
    $AptitudActitud = AptitudActitud::model()->find($criteria);
    $Competencia = Competencia::model()->find($criteria);
    $Desempenho = Desempenho::model()->find($criteria);
```

```
$Habilidad = Habilidad::model()->find($criteria);

$Fisica = Fisica::model()->find($criteria);

$Medico = ExamenMedico::model()->find($criteria);

$Antidoping = Antidoping::model()->find($criteria);

$this->render('view',array(

    'model'=>$model,'Espiritu_bomb'=>$Espiritu_bomb,'AptitudActitud'=>$AptitudActitud,
    'Competencia'=>$Competencia,'Desempenho'=>$Desempenho,
    'Habilidad'=>$Habilidad,'Fisica'=>$Fisica,
    'Medico'=>$Medico,'Antidoping'=>$Antidoping,

));

}

/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function actionCreate($cedula)
{
    $model = new Postulados;

    $model_evaluador = new Evaluador;

    //print_r($cedula);

    //yii::app()->end();

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['Postulados']))
    {
```

```
$criteria = new CDbCriteria;

$criteria->addCondition('cedula=:ced');

$criteria->params=array(':ced'=>$cedula);

$postulado = Postularse::model()->find($criteria);

$funcionario = Funcionarios::model()->find($criteria);

$model->id_conf_asc_fecha = $postulado->id_conf_asc_fecha;

$model->Cedula=$funcionario->Cedula;

$model->fecha_postulacion = date("Y-m-d");

$model->hora_postulacion = date('h:i:s');

$model->cedula_fun_recepcion = $_POST['Postulados']['cedula_fun_recepcion'];

$model->des_datos_receptor = $_POST['Postulados']['des_datos_receptor'];

$model->Cod_Jerarquia = $funcionario->Cod_Jerarquia;

$model->id_asc_fecha = 1010; //No se que datos van aquí

$model->id_fecha_ingreso = 1010; //Prueba

// $model->id_fecha_ingreso = $model->getFechaIngreso($cedula);

$model->num_modificar = 1; //No se que datos van aquí

//Cambiar el status del postulado

$postulado->id_status = 1;

$postulado->save();

//Crear relacion evaluador - evaluado

$model_evaluador->Cedula = $cedula;

$model_evaluador->CedEvaluador =

$_POST['Postulados']['cedula_fun_recepcion'];

$model_evaluador->id_conf_asc_fecha = $model->getConfAscFecha()-

>id_conf_asc_fecha;

$model_evaluador->save();
```

```
//Asignar rol a evaluador

$Evaluador = $model->getUser($_POST['Postulados']['cedula_fun_recepcion']);

if($Evaluador)
{
    if(yii::app()->authManager->checkAccess('demo',$Evaluador-
>id))//Verifica que tenga permisos de usuario
    {
        yii::app()->authManager->revoke('demo',$Evaluador-
>id);//Quita el items

        yii::app()->authManager->assign('Evaluador',$Evaluador-
>id);//asigna el items
    }
}

//$model->attributes=$_POST['Postulados'];

if($model->save())

    $this->redirect(array('view','id'=>$model->id_postulados));

}

$this->render('create',array(

    'model'=>$model, 'cedula' => $cedula,

));

}

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */
public function actionUpdate($id)
{

    $model=$this->loadModel($id);
```

```
// Uncomment the following line if AJAX validation is needed
// $this->performAjaxValidation($model);

if(isset($_POST['Postulados']))
{
    $model->attributes=$_POST['Postulados'];
    if($model->save())
        $this->redirect(array('view','id'=>$model->id_postulados));
}

$this->render('update',array(
    'model'=>$model,
));
}

/**
 * Deletes a particular model.
 * If deletion is successful, the browser will be redirected to the 'admin' page.
 * @param integer $id the ID of the model to be deleted
 */
public function actionDelete($id)
{
    $this->loadModel($id)->delete();

    // if AJAX request (triggered by deletion via admin grid view), we should not redirect the
    browser
    if(!isset($_GET['ajax']))
        $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
```

```
}

/**
 * Lists all models.
 */
public function actionIndex()
{
    $model = new Postulados;
    $criteria = new CDbCriteria;

    $criteria->addCondition('id_conf_asc_fecha=:fecha');
    if($model->getConfAscFecha())
        $criteria->params=array(':fecha'=>$model->getConfAscFecha()-
>id_conf_asc_fecha);
    else
        $criteria->params=array(':fecha'=>0);

    $dataProvider=new CActiveDataProvider('Postulados');
    $this->render('index',array(
        'dataProvider'=>$dataProvider,
    ));
}

public function actionPostulados()
{
    $model = new Postulados;
    $criteria = new CDbCriteria;

    $criteria->addCondition('cedula_fun_recepcion=:ced');
    $criteria->addCondition('id_conf_asc_fecha=:fecha_asc');
    if($model->getConfAscFecha())
```

```
$criteria->params=array(':ced'=>yii::app()->user->getState('cedula'),':fecha_asc'=>$model-  
>getConfAscFecha()->id_conf_asc_fecha);
```

```
else
```

```
$criteria->params=array(':ced'=>yii::app()->user->getState('cedula'),':fecha_asc'=>0);
```

```
$dataProvider=new CActiveDataProvider('Postulados', array(  
    'criteria'=>$criteria,  
));  
  
$this->render('index',array(  
    'dataProvider'=>$dataProvider,  
));  
}
```

```
/**
```

```
 * Manages all models.
```

```
 */
```

```
public function actionAdmin()
```

```
{
```

```
    $model=new Postulados('search');
```

```
    $model->unsetAttributes(); // clear any default values
```

```
    if(isset($_GET['Postulados']))
```

```
        $model->attributes=$_GET['Postulados'];
```

```
    $this->render('admin',array(  
        'model'=>$model,  
    ));  
}
```

```
/**
 * Returns the data model based on the primary key given in the GET variable.
 * If the data model is not found, an HTTP exception will be raised.
 * @param integer $id the ID of the model to be loaded
 * @return Postulados the loaded model
 * @throws CHttpException
 */
public function loadModel($id)
{
    $model=Postulados::model()->findByPk($id);
    if($model===null)
        throw new CHttpException(404,'The requested page does not exist.');
```



```
    return $model;
}

/**
 * Performs the AJAX validation.
 * @param Postulados $model the model to be validated
 */
protected function performAjaxValidation($model)
{
    if(isset($_POST['ajax']) && $_POST['ajax']=='postulados-form')
    {
        echo CActiveForm::validate($model);
        Yii::app()->end();
    }
}
}
```

Controlador ConfigPlaza

<?php

```
class ConfigPlazaController extends Controller
```

```
{
```

```
    /**
```

```
     * @var string the default layout for the views. Defaults to '//layouts/column2', meaning
```

```
     * using two-column layout. See 'protected/views/layouts/column2.php'.
```

```
    */
```

```
    public $layout='//layouts/column2';
```

```
    /**
```

```
     * @return array action filters
```

```
    */
```

```
    public function filters()
```

```
    {
```

```
        return array(
```

```
            'accessControl', // perform access control for CRUD operations
```

```
            'postOnly + delete', // we only allow deletion via POST request
```

```
        );
```

```
    }
```

```
    /**
```

```
     * Specifies the access control rules.
```

```
     * This method is used by the 'accessControl' filter.
```

```
     * @return array access control rules
```

```
    */
```

```
    public function accessRules()
```

```
    {
```

```
return array(

    array('allow', // allow all users to perform 'index' and 'view' actions
        'actions'=>array('index','view'),
        'users'=>array('*'),
    ),
    array('allow', // allow authenticated user to perform 'create' and 'update' actions
        'actions'=>array('create','update'),
        'users'=>array('@'),
    ),
    array('allow', // allow admin user to perform 'admin' and 'delete' actions
        'actions'=>array('admin','delete'),
        'users'=>array('admin'),
    ),
    array('deny', // deny all users
        'users'=>array('*'),
    ),
);

}

/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $this->render('view',array(
        'model'=>$this->loadModel($id),
    ));
}
```

```
/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function actionCreate()
{
    $model=new ConfigPlaza;

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['ConfigPlaza']))
    {
        $model->attributes=$_POST['ConfigPlaza'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id_config_nota_plaza));
    }

    $this->render('create',array(
        'model'=>$model,
    ));
}

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */
```

```
public function actionUpdate($id)
{
    $model=$this->loadModel($id);

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['ConfigPlaza']))
    {
        $model->attributes=$_POST['ConfigPlaza'];
        if($model->save())
            $this->redirect(array('view','id'=>$model->id_config_nota_plaza));
    }

    $this->render('update',array(
        'model'=>$model,
    ));
}

/**
 * Deletes a particular model.
 * If deletion is successful, the browser will be redirected to the 'admin' page.
 * @param integer $id the ID of the model to be deleted
 */
public function actionDelete($id)
{
    $this->loadModel($id)->delete();
```

browser // if AJAX request (triggered by deletion via admin grid view), we should not redirect the

```
if(!isset($_GET['ajax']))
    $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] : array('admin'));
}

/**
 * Lists all models.
 */
public function actionIndex()
{
    $dataProvider=new CActiveDataProvider('ConfigPlaza');
    $this->render('index',array(
        'dataProvider'=>$dataProvider,
    ));
}

/**
 * Manages all models.
 */
public function actionAdmin()
{
    $model=new ConfigPlaza('search');
    $model->unsetAttributes(); // clear any default values
    if(isset($_GET['ConfigPlaza']))
        $model->attributes=$_GET['ConfigPlaza'];

    $this->render('admin',array(
        'model'=>$model,
```

```
    ));  
}  
  
/**  
 * Returns the data model based on the primary key given in the GET variable.  
 * If the data model is not found, an HTTP exception will be raised.  
 * @param integer $id the ID of the model to be loaded  
 * @return ConfigPlaza the loaded model  
 * @throws CHttpException  
 */  
public function loadModel($id)  
{  
    $model=ConfigPlaza::model()->findByPk($id);  
    if($model===null)  
        throw new CHttpException(404,'The requested page does not exist.');
```

```
    return $model;  
}  
  
/**  
 * Performs the AJAX validation.  
 * @param ConfigPlaza $model the model to be validated  
 */  
protected function performAjaxValidation($model)  
{  
    if(isset($_POST['ajax']) && $_POST['ajax']=='config-plaza-form')  
    {  
        echo CActiveForm::validate($model);  
        Yii::app()->end();  
    }  
}
```

```
}  
  
}
```

Controlador FechaAsc

<?php

```
class FechaAscController extends Controller  
{  
    /**  
     * @var string the default layout for the views. Defaults to '//layouts/column2',  
meaning  
     * using two-column layout. See 'protected/views/layouts/column2.php'.  
     */  
    public $layout='//layouts/column2';  
  
    /**  
     * @return array action filters  
     */  
    public function filters()  
    {  
        return array(  
            'accessControl', // perform access control for CRUD operations  
            'postOnly + delete', // we only allow deletion via POST request  
        );  
    }  
  
    /**
```

* Specifies the access control rules.

* This method is used by the 'accessControl' filter.

* @return array access control rules

*/

```
public function accessRules()
{
    return array(
        array('allow', // allow all users to perform 'index' and 'view' actions
            'actions'=>array('index','view'),
            'users'=>array('*'),
        ),
        array('allow', // allow authenticated user to perform 'create' and
'update' actions
            'actions'=>array('create','update'),
            'users'=>array('@'),
        ),
        array('allow', // allow admin user to perform 'admin' and 'delete'
actions
            'actions'=>array('admin','delete'),
            'roles'=>array('admin'),
        ),
        array('deny', // deny all users
            'users'=>array('*'),
        ),
    );
}
```

```
/**
 * Displays a particular model.
 * @param integer $id the ID of the model to be displayed
 */
public function actionView($id)
{
    $this->render('view',array(
        'model'=>$this->loadModel($id),
    ));
}

/**
 * Creates a new model.
 * If creation is successful, the browser will be redirected to the 'view' page.
 */
public function actionCreate()
{
    $model=new FechaAsc;

    // Uncomment the following line if AJAX validation is needed
    // $this->performAjaxValidation($model);

    if(isset($_POST['FechaAsc']))
    {
        $model->attributes=$_POST['FechaAsc'];
```

```
        if($model->save())

            $this->redirect(array('view','id'=>$model-
>id_conf_asc_fecha));

        }

        $this->render('create',array(

            'model'=>$model,

        ));

    }

/**
 * Updates a particular model.
 * If update is successful, the browser will be redirected to the 'view' page.
 * @param integer $id the ID of the model to be updated
 */

    public function getUltimaFecha()

    {

        $criteria = new CDbCriteria;

        $criteria->addCondition('"des_proceso_asc"='EN PROCESO"', 'OR');

        $criteria->addCondition('"des_estatus_cond"='ACTIVO"', 'OR');

        $Fecha_asc = FechaAsc::model()->find($criteria);

        if($Fecha_asc)

            return true;//return $Fecha_asc->check_in_range($Fecha_asc->fecha_postulacion,
$Fecha_asc->fecha_proceso_asc, date('Y-m-d'));
```

```
        else

            return false;

    }

    public function actionUpdate($id)
    {

        $model=$this->loadModel($id);

        // Uncomment the following line if AJAX validation is needed
        // $this->performAjaxValidation($model);

        if(isset($_POST['FechaAsc']))
        {

            $model->attributes=$_POST['FechaAsc'];

                                if($model->save())
                                $this->redirect(array('view','id'=>$model-
>id_conf_asc_fecha));
        }

        $this->render('update',array(

            'model'=>$model,

        ));

    }

    /**
```

-
- * Deletes a particular model.
 - * If deletion is successful, the browser will be redirected to the 'admin' page.
 - * @param integer \$id the ID of the model to be deleted
 - */

```
public function actionDelete($id)
```

```
{
```

```
    $this->loadModel($id)->delete();
```

```
    // if AJAX request (triggered by deletion via admin grid view), we should not  
    redirect the browser
```

```
    if(!isset($_GET['ajax']))
```

```
        // $this->redirect(isset($_POST['returnUrl']) ? $_POST['returnUrl'] :  
        array('admin'));
```

```
        $this->redirect(array('index'));
```

```
}
```

```
/**
```

- * Lists all models.

```
*/
```

```
public function actionIndex()
```

```
{
```

```
    $dataProvider=new CActiveDataProvider('FechaAsc');
```

```
    $this->render('index',array(
```

```
        'dataProvider'=>$dataProvider,
```

```
    ));
```

```
}
```

```
/**
 * Manages all models.
 */
public function actionAdmin()
{
    $model=new FechaAsc('search');
    $model->unsetAttributes(); // clear any default values
    if(isset($_GET['FechaAsc']))
        $model->attributes=$_GET['FechaAsc'];

    $this->render('admin',array(
        'model'=>$model,
    ));
}

/**
 * Returns the data model based on the primary key given in the GET variable.
 * If the data model is not found, an HTTP exception will be raised.
 * @param integer $id the ID of the model to be loaded
 * @return FechaAsc the loaded model
 * @throws CHttpException
 */
public function loadModel($id)
{
    $model=FechaAsc::model()->findByPk($id);
```

```
        if($model===null)

            throw new CHttpException(404,'The requested page does not
exist.');
```

```
        return $model;

    }

    /**
     * Performs the AJAX validation.
     * @param FechaAsc $model the model to be validated
     */
    protected function performAjaxValidation($model)
    {
        if(isset($_POST['ajax']) && $_POST['ajax']=='fecha-asc-form')
        {
            echo CActiveForm::validate($model);
            Yii::app()->end();
        }
    }
}
```

Clase FechaAsc

```
<?php
```

```
/**
 * This is the model class for table "tbl_asc_config_fecha_asc".
 *
 * The followings are the available columns in table 'tbl_asc_config_fecha_asc':
```

```
* @property integer $id_conf_asc_fecha
* @property string $fecha_proceso_asc
* @property string $des_proceso_asc
* @property string $des_estatus_cond
* @property string $fecha_postulacion
*/

class FechaAsc extends CActiveRecord
{
    /**
     * @return string the associated database table name
     */
    public function tableName()
    {
        return 'tbl_asc_config_fecha_asc';
    }

    /**
     * @return array validation rules for model attributes.
     */
    public function rules()
    {
        // NOTE: you should only define rules for those attributes that
        // will receive user inputs.
        return array(
            array('des_estatus_cond, fecha_proceso_asc, fecha_postulacion',
                'required'),
```

```
        array('id_conf_asc_fecha', 'numerical', 'integerOnly'=>true),
        array('des_proceso_asc, des_estatus_cond', 'length', 'max'=>15),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be
searched.

        array('id_conf_asc_fecha, fecha_proceso_asc, des_proceso_asc,
des_estatus_cond, fecha_postulacion', 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(
    );
}

/**
 * @return array customized attribute labels (name=>label)
 */
public function attributeLabels()
{
```

```
        return array(

            'id_conf_asc_fecha' => 'Id Conf Asc Fecha',

            'fecha_proceso_asc' => 'Fecha Proceso Asc',

            'des_proceso_asc' => 'Des Proceso Asc',

            'des_estatus_cond' => 'Des Estatus Cond',

            'fecha_postulacion' => 'Fecha Postulacion',

        );

    }

    /**
     * Retrieves a list of models based on the current search/filter conditions.
     *
     * Typical usecase:
     * - Initialize the model fields with values from filter form.
     * - Execute this method to get CActiveDataProvider instance which will filter
     *   models according to data in model fields.
     * - Pass data provider to CGridView, CListView or any similar widget.
     *
     * @return CActiveDataProvider the data provider that can return the models
     *   based on the search/filter conditions.
     */
    public function search()
    {

        // @todo Please modify the following code to remove attributes that
        should not be searched.
```

```
$criteria=new CDbCriteria;
```

```
$criteria->compare('id_conf_asc_fecha',$this->id_conf_asc_fecha);  
$criteria->compare('fecha_proceso_asc',$this->fecha_proceso_asc,true);  
$criteria->compare('des_proceso_asc',$this->des_proceso_asc,true);  
$criteria->compare('des_estatus_cond',$this->des_estatus_cond,true);  
$criteria->compare('fecha_postulacion',$this->fecha_postulacion,true);
```

```
return new CActiveDataProvider($this, array(  
    'criteria'=>$criteria,  
));
```

```
}
```

```
/**
```

```
 * Returns the static model of the specified AR class.
```

```
 * Please note that you should have this exact method in all your CActiveRecord  
descendants!
```

```
 * @param string $className active record class name.
```

```
 * @return FechaAsc the static model class
```

```
 */
```

```
public static function model($className=__CLASS__)
```

```
{
```

```
    return parent::model($className);
```

```
}
```

```
public function getDescripcion(){
```

```
        return $this->fecha_proceso_asc.' '.$this->des_proceso_asc;
    }

    public function getLastPostulationDate()
    {
        $criteria = new CDbCriteria;
        $criteria->order = 'fecha_postulacion DESC';
        $LastDate = FechaAsc::model()->find($criteria);

        if($LastDate)
        {
            $year = date('Y', strtotime($LastDate->fecha_postulacion));

            if($year == date('Y'))
                return $LastDate;
            else
                return null;
        }
        else
            return null;
    }

    public function getLastPromotionDate()
    {
        $criteria = new CDbCriteria;
        $criteria->order = 'fecha_proceso_asc DESC';
```

```
$LastDate = FechaAsc::model()->find($criteria);
```

```
if($LastDate)
```

```
{
```

```
    $year = date('Y', strtotime($LastDate->fecha_proceso_asc));
```

```
    if($year == date('Y'))
```

```
        return $LastDate;
```

```
    else
```

```
        return null;
```

```
}
```

```
else
```

```
    return null;
```

```
}
```

```
public function check_in_range($start_date, $end_date, $evaluate)
```

```
{
```

```
    $start_ts = strtotime($start_date);
```

```
    $end_ts = strtotime($end_date);
```

```
    $user_ts = strtotime($evaluate);
```

```
    return (($user_ts >= $start_ts) && ($user_ts <= $end_ts));
```

```
}
```

```
public static function obtenerUltimoid(){
```

```
    $sql = "SELECT MAX(`id_conf_asc_fecha`) AS ultimo FROM  
`tbl_asc_config_fecha_asc`";
```

```
$num=yii::app()->db->createCommand($sql)->queryAll();  
if($num===null)  
    throw new CHttpException(404,' no se puede procesar.');
```

```
return $num[0]['ultimo'];  
}  
}
```

Clase Postularse

```
<?php
```

```
/**
```

```
* Este es el modelo de clase por la tabla "tbl_postularse".
```

```
*
```

```
* Los siguientes son columnas disponibles de la tabla 'tbl_postularse':
```

```
* @property integer $id_postularse
```

```
* @property integer $cedula
```

```
* @property string $nombre
```

```
* @property string $apellido
```

```
* @property string $cargo
```

```
* @property integer $id_status
```

```
*/
```

```
class Postularse extends CActiveRecord
```

```
{
```

```
    /**
```

```
    * @return string asociado al nombre de la tabla.
```

```
    */
```

```
    public function tableName()
```

```
{  
  
    return 'tbl_postularse';  
  
}  
  
/**  
 * @return arreglo de regla de validaciones para los atributos del modelo.  
 */  
public function rules()  
{  
    // NOTE: Deben definirse reglas para esos atributos.  
    // recibiras inputs de usuarios.  
    return array(  
        array('cedula', 'required'),  
        //array('cedula','compare','compareValue'=>yii::app()->user-  
>getState('cedula'),'operator'=>'==','message'=>'Cédula no es valida. Ingrese su cédula  
para postularse.'),  
        //array('cedula','unique'),  
        array('cedula', 'numerical', 'integerOnly'=>true),  
        array('nombre, apellido, cargo', 'length', 'max'=>30),  
        // la siguiente regla es usada para búsqueda.  
        array('id_postularse, cedula, nombre, apellido, cargo', 'safe',  
'on'=>'search'),  
    );  
}  
  
/**  
 * @return array reglas relacionales.
```

```
*/  
  
public function relations()  
{  
    return array(  
    );  
}  
  
/**  
 * @return array de etiquetas de atributos configuradas (name=>label)  
 */  
  
public function attributeLabels()  
{  
    return array(  
        'id_postularse' => 'Id Postularse',  
        'cedula' => 'Cedula',  
        'nombre' => 'Nombre',  
        'apellido' => 'Apellido',  
        'cargo' => 'Cargo',  
        'id_status' => 'Id Status',  
        'id_conf_asc_fecha' => 'Id Conf Asc Fecha',  
    );  
}  
  
/**  
 * @return boolean indicando si existe un funcionario postulado o no.  
 */
```

```
public function getFuncionarioPost()
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('cedula=:ced');
    $criteria->params=array(':ced'=>yii::app()->user->getState('cedula'));
    $Funcionario = Postularse::model()->find($criteria);
    if($Funcionario)
        return true;
    elseif($Funcionario == null)
        return false;
}

/**
 * @return string con el estado de la solicitud.
 */
public function getStatus()
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('cedula=:ced');
    $criteria->params=array(':ced'=>yii::app()->user->getState('cedula'));
    $Funcionario = Postularse::model()->find($criteria);
    if($Funcionario)
        return $Funcionario->id_status;
    elseif($Funcionario == null)
        return null;
}
```

```
/**
 * @return modelo FechaAsc que esté activo o en proceso.
 */
public function getFechaAsc()
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('des_proceso_asc=:des1', 'OR');
    $criteria->addCondition('des_estatus_cond=:des2', 'OR');
    $criteria->params=array(':des1'=>'EN PROCESO', ':des2'=>'ACTIVO');
    $Fecha_Asc = FechaAsc::model()->find($criteria);

    if($Fecha_Asc)
        return $Fecha_Asc;
    else
        return null;
}

/**
 * @return modelo Postularse con la cedula especificada.
 */
public function getFuncionario()
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('cedula=:ced');
    $criteria->params=array(':ced'=>yii::app()->user->getState('cedula'));
```

```
$criteria->order ='id_postularse DESC';

$criteria->limit = 1;

$Funcionario = Postularse::model()->find($criteria);

if($Funcionario)

    return $Funcionario;

elseif($Funcionario == null)

    return null;

}

/**
 * @return
 */

public function getFuncionarioCed($cedula)

{

    $criteria = new CDbCriteria;

$criteria->addCondition('cedula=:ced');

$criteria->params=array(':ced'=>$cedula);

$Funcionario = Funcionarios::model()->find($criteria);

if($Funcionario)

    return $Funcionario;

elseif($Funcionario == null)

    return null;

}

public function getJerarquiaName($cod)
```

```
{  
  
    $criteria = new CDbCriteria;  
  
    $criteria->addCondition('Cod_Jerarquia=:cod');  
    $criteria->params=array(':cod'=>$cod);  
    $Funcionario = Jerarquia::model()->find($criteria);  
  
    if($Funcionario)  
        return $Funcionario->Descripcion_Jerarquia;  
    elseif(is_null($Funcionario))  
        return " ";  
}  
  
public function getPostulante($cedula)  
{  
  
    $criteria = new CDbCriteria;  
  
    $criteria->addCondition('cedula=:ced');  
    $criteria->params=array(':ced'=>$cedula);  
    $Funcionario = Postularse::model()->find($criteria);  
    if($Funcionario)  
        return $Funcionario;  
    elseif($Funcionario == null)  
        return null;  
}  
  
public function check_in_range($start_date, $end_date, $evaluame)  
{
```

```
$start_ts = strtotime($start_date);  
$end_ts = strtotime($end_date);  
$user_ts = strtotime($evaluame);  
return (($user_ts >= $start_ts) && ($user_ts <= $end_ts));  
}
```

```
public function getUltimaFecha($cedula)  
{  
    $criteria = new CDbCriteria;  
    $criteria->addCondition('cedula=:ced');  
    $criteria->order = 'fecha_ascenso DESC';  
    $criteria->limit = 1;  
    $criteria->params=array(':ced'=>$cedula);  
  
    $FechaAscenso = FechaAscenso::model()->find($criteria);  
    if($FechaAscenso)  
        return $FechaAscenso->fecha_ascenso;  
    else  
        return $this->getFechaIngreso($cedula);  
}
```

```
public function getFechaIngreso($cedula)  
{  
    $criteria = new CDbCriteria;  
    $criteria->addCondition('cedula=:ced');  
    $criteria->params=array(':ced'=>$cedula);
```

```
$Fecha_Ingreso = FechaIngreso::model()->find($criteria);
```

```
return $Fecha_Ingreso->id_fecha_ingreso;  
}
```

```
public function getMaximoRango($Funcionario)  
{
```

```
    $MaximoRango = 0;
```

```
    if($Funcionario->num_categoria == 1)
```

```
        $MaximoRango = 0;
```

```
    if($Funcionario->num_categoria == 2)
```

```
        $MaximoRango = 1;
```

```
    if($Funcionario->num_categoria == 1)
```

```
        $MaximoRango = 3;
```

```
    return $MaximoRango;
```

```
}
```

```
public function anhosRequeridoJerarquia($numero)#años de jerarquia
```

```
{
```

```
    if($numero == -1)
```

```
        $numero = 0;
```

```
    $anhos=array(  
        '0'=>'3',
```

```
        #3 años para comandante general
```

```
        '1'=>'3', #3 años para coronel
```

```
'2'=>'3', #3 años teniente coronel
'3'=>'3', #3 años mayor
'4'=>'2', #2 años para capitán
'5'=>'2', #2 años para teniente
'6'=>'2', #2 años para subteniente
'7'=>'1', #1 año para Sargento Ayudante
'8'=>'1', #1 año para Sargento primero
'9'=>'1', #1 año para Sargento Segundo
'10'=>'1', #1 año para Cabo Primero
'11'=>'1', #1 año para Cabo Segundo
'12'=>'1', #1 año para Distinguido
);
return $anhos[$numero];
}

/* public function getStatusName($id)
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('id_status=:id');
    $criteria->params=array(':id'=>$id);
    $status = Status::model()->find($criteria);
    return $status;
}

*/

/**
```

-
- * Retrieves a list of models based on the current search/filter conditions.
 - *
 - * Typical usecase:
 - * - Initialize the model fields with values from filter form.
 - * - Execute this method to get CActiveDataProvider instance which will filter
 - * models according to data in model fields.
 - * - Pass data provider to CGridView, CListView or any similar widget.
 - *
 - * @return CActiveDataProvider the data provider that can return the models
 - * based on the search/filter conditions.
 - */

```
public function search()
```

```
{
```

```
    // @todo Please modify the following code to remove attributes that  
    should not be searched.
```

```
    $criteria=new CDbCriteria;
```

```
    $criteria->compare('id_postularse',$this->id_postularse);
```

```
    $criteria->compare('cedula',$this->cedula);
```

```
    $criteria->compare('nombre',$this->nombre,true);
```

```
    $criteria->compare('apellido',$this->apellido,true);
```

```
    $criteria->compare('cargo',$this->cargo,true);
```

```
    $criteria->compare('id_status',$this->id_status);
```

```
    $criteria->compare('id_conf_asc_fecha',$this->id_conf_asc_fecha);
```

```
        return new CActiveDataProvider($this, array(
            'criteria'=>$criteria,
        ));
    }

    /**
     * Returns the static model of the specified AR class.
     * Please note that you should have this exact method in all your CActiveRecord
    descendants!
     * @param string $className active record class name.
     * @return Postularse the static model class
     */
    public static function model($className=__CLASS__)
    {
        return parent::model($className);
    }
}
```

Clase ConfigPlaza

```
<?php

/**
 * This is the model class for table "tbl_asc_config_nota_plaza".
 *
 * The followings are the available columns in table 'tbl_asc_config_nota_plaza':
 * @property integer $id_config_nota_plaza
 * @property integer $Cod_Jerarquia
```

```
* @property integer $num_plazas
* @property string $num_nota_minima
* @property string $num_nota_final
* @property string $des_orden_general
* @property string $des_jerarq_orden
* @property integer $id_conf_asc_fecha
* @property integer $num_tiempo_jerar
*/
```

```
class ConfigPlaza extends CActiveRecord
```

```
{
```

```
    /**
```

```
     * @return string the associated database table name
```

```
    */
```

```
    public function tableName()
```

```
    {
```

```
        return 'tbl_asc_config_nota_plaza';
```

```
    }
```

```
    /**
```

```
     * @return array validation rules for model attributes.
```

```
    */
```

```
    public function rules()
```

```
    {
```

```
        // NOTE: you should only define rules for those attributes that
```

```
        // will receive user inputs.
```

```
        return array(
```

```
        array('Cod_Jerarquia, id_conf_asc_fecha, num_tiempo_jerar',
'id_conf_asc_fecha, num_tiempo_jerar', 'numerical', 'integerOnly'=>true),
        array('num_nota_minima, num_nota_final', 'length', 'max'=>18),
        array('des_orden_general, des_jerarq_orden', 'length', 'max'=>255),
        // The following rule is used by search().
        // @todo Please remove those attributes that should not be
searched.

        array('id_config_nota_plaza, Cod_Jerarquia, num_plazas,
num_nota_minima, num_nota_final, des_orden_general, des_jerarq_orden,
id_conf_asc_fecha, num_tiempo_jerar', 'safe', 'on'=>'search'),
    );
}

/**
 * @return array relational rules.
 */
public function relations()
{
    // NOTE: you may need to adjust the relation name and the related
    // class name for the relations automatically generated below.
    return array(
    );
}

/**
```

```
* @return array customized attribute labels (name=>label)
*/
public function attributeLabels()
{
    return array(
        'id_config_nota_plaza' => 'Id Config Nota Plaza',
        'Cod_Jerarquia' => 'Cod Jerarquia',
        'num_plazas' => 'Num Plazas',
        'num_nota_minima' => 'Num Nota Minima',
        'num_nota_final' => 'Num Nota Final',
        'des_orden_general' => 'Des Orden General',
        'des_jerarq_orden' => 'Des Jerarq Orden',
        'id_conf_asc_fecha' => 'Id Conf Asc Fecha',
        'num_tiempo_jerar' => 'Num Tiempo Jerar',
    );
}

public function listJerarquias()
{
    return CHtml::listData(Jerarquia::model()-
>findAll(),'Cod_Jerarquia','Descripcion_Jerarquia') ;
}

public function listAscFechas()
{
```

```
        return CHtml::listData(FechaAsc::model()-
>findAll(),'id_conf_asc_fecha','fecha_postulacion') ;
    }

/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
 * - Execute this method to get CActiveDataProvider instance which will filter
 * models according to data in model fields.
 * - Pass data provider to CGridView, CListView or any similar widget.
 *
 * @return CActiveDataProvider the data provider that can return the models
 * based on the search/filter conditions.
 */
public function search()
{
    // @todo Please modify the following code to remove attributes that
    should not be searched.
```

```
$criteria=new CDbCriteria;
```

```
$criteria->compare('id_config_nota_plaza',$this->id_config_nota_plaza);
```

```
$criteria->compare('Cod_Jerarquia',$this->Cod_Jerarquia);
```

```
$criteria->compare('num_plazas',$this->num_plazas);
```

```
$criteria->compare('num_nota_minima',$this->num_nota_minima,true);
$criteria->compare('num_nota_final',$this->num_nota_final,true);
$criteria->compare('des_orden_general',$this->des_orden_general,true);
$criteria->compare('des_jerarq_orden',$this->des_jerarq_orden,true);
$criteria->compare('id_conf_asc_fecha',$this->id_conf_asc_fecha);
$criteria->compare('num_tiempo_jerar',$this->num_tiempo_jerar);

return new CActiveDataProvider($this, array(
    'criteria'=>$criteria,
));
}

/**
 * Returns the static model of the specified AR class.
 * Please note that you should have this exact method in all your CActiveRecord
descendants!
 * @param string $className active record class name.
 * @return ConfigPlaza the static model class
 */
public static function model($className=__CLASS__)
{
    return parent::model($className);
}
}
```

Clase Postulados

```
<?php
```

```
/**
```

```
 * This is the model class for table "tbl_asc_postulados".
```

```
 *
```

```
 * The followings are the available columns in table 'tbl_asc_postulados':
```

```
 * @property integer $id_postulados
```

```
 * @property integer $id_conf_asc_fecha
```

```
 * @property double $Cedula
```

```
 * @property string $fecha_postulacion
```

```
 * @property string $hora_postulacion
```

```
 * @property double $cedula_fun_recepcion
```

```
 * @property string $des_datos_receptor
```

```
 * @property integer $Cod_Jerarquia
```

```
 * @property integer $id_asc_fecha
```

```
 * @property integer $id_fecha_ingreso
```

```
 * @property integer $num_modificar
```

```
 */
```

```
class Postulados extends CActiveRecord
```

```
{
```

```
    /**
```

```
     * @return string the associated database table name
```

```
    */
```

```
    public function tableName()
```

```
    {
```

```
        return 'tbl_asc_postulados';
    }

    /**
     * @return array validation rules for model attributes.
     */
    public function rules()
    {
        // NOTE: you should only define rules for those attributes that
        // will receive user inputs.
        return array(
            array('cedula_fun_recepcion, des_datos_receptor', 'required'),
            array('id_postulados, id_conf_asc_fecha, Cod_Jerarquia,
id_asc_fecha, id_fecha_ingreso, num_modificar', 'numerical', 'integerOnly'=>true),
            array('Cedula, cedula_fun_recepcion', 'numerical'),
            array('des_datos_receptor', 'length', 'max'=>100),
            // The following rule is used by search().
            // @todo Please remove those attributes that should not be
searched.

            array('id_postulados, id_conf_asc_fecha, Cedula, fecha_postulacion,
hora_postulacion, cedula_fun_recepcion, des_datos_receptor, Cod_Jerarquia,
id_asc_fecha, id_fecha_ingreso, num_modificar', 'safe', 'on'=>'search'),
        );
    }

    /**
     * @return array relational rules.
     */
```

```
*/  
  
public function relations()  
{  
  
    // NOTE: you may need to adjust the relation name and the related  
    // class name for the relations automatically generated below.  
  
    return array(  
  
    );  
  
}  
  
/**  
  
 * @return array customized attribute labels (name=>label)  
 */  
  
public function attributeLabels()  
{  
  
    return array(  
  
        'id_postulados' => 'Id Postulados',  
        'id_conf_asc_fecha' => 'Id Conf Asc Fecha',  
        'Cedula' => 'Cedula',  
        'fecha_postulacion' => 'Fecha Postulacion',  
        'hora_postulacion' => 'Hora Postulacion',  
        'cedula_fun_recepcion' => 'Cedula Fun Recepcion',  
        'des_datos_receptor' => 'Des Datos Receptor',  
        'Cod_Jerarquia' => 'Cod Jerarquia',  
        'id_asc_fecha' => 'Id Asc Fecha',  
        'id_fecha_ingreso' => 'Id Fecha Ingreso',  
        'num_modificar' => 'Num Modificar',
```

```
    );  
}  
  
public function getConfAscFecha()  
{  
    $criteria = new CDbCriteria;  
    $criteria->addCondition('des_proceso_asc=:des1', 'OR');  
    $criteria->addCondition('des_estatus_cond=:des2', 'OR');  
    $criteria->params=array(':des1'=>'EN PROCESO', ':des2'=>'ACTIVO');  
    $Fecha_Asc = FechaAsc::model()->find($criteria);  
  
    return $Fecha_Asc;  
}  
  
public function listFuncionarios($cedula)  
{  
    $criteria = new CDbCriteria;  
    $criteria->addCondition('NOT Cedula=:ced');  
    $criteria->params=array(':ced'=>$cedula);  
    return CHtml::listData(Funcionarios::model()-  
>findAll($criteria),'Cedula','NombreCedula') ;  
}  
  
public function getFechaIngreso($cedula)  
{  
    $criteria = new CDbCriteria;
```

```
$criteria->addCondition('cedula=:ced');  
$criteria->params=array(':ced'=>$cedula);  
$Fecha_Ingreso = FechaIngreso::model()->find($criteria);  
  
return $Fecha_Ingreso->id_fecha_ingreso;  
}  
/*  
*Funcion que recibe un parametro cedula*  
* y realiza una busqueda en la bd , *  
*encontrando coincidencia con la cedula *  
* del funcionario *  
* retornando la cedula del funcionario *  
* de recepcion sino retorna null *  
*/  
public function getFuncionarioRecepcion($cedula)  
{  
    $criteria = new CDbCriteria;  
    $criteria->addCondition('cedula=:ced');  
    $criteria->params=array(':ced'=>$cedula);  
    $funcionario = Postulados::model()->find($criteria);  
    if($funcionario)  
        return $funcionario->cedula_fun_recepcion;  
    elseif($funcionario == null)  
        return null;  
}
```

```
public function getUser($cedula)
{
    $criteria = new CDbCriteria;
    $criteria->addCondition('cedula=:ced');
    $criteria->params=array(':ced'=>$cedula);
    $Usuario = Usuario::model()->find($criteria);
    if($Usuario)
        return $Usuario;
    elseif($Usuario == null)
        return null;
}

/**
 * Retrieves a list of models based on the current search/filter conditions.
 *
 * Typical usecase:
 * - Initialize the model fields with values from filter form.
 * - Execute this method to get CActiveDataProvider instance which will filter
 *   models according to data in model fields.
 * - Pass data provider to CGridView, CListView or any similar widget.
 *
 * @return CActiveDataProvider the data provider that can return the models
 *   based on the search/filter conditions.
 */
public function search()
```

```
{
```

```
    // @todo Please modify the following code to remove attributes that  
    should not be searched.
```

```
    $criteria=new CDbCriteria;
```

```
    $criteria->compare('id_postulados',$this->id_postulados);
```

```
    $criteria->compare('id_conf_asc_fecha',$this->id_conf_asc_fecha);
```

```
    $criteria->compare('Cedula',$this->Cedula);
```

```
    $criteria->compare('fecha_postulacion',$this->fecha_postulacion,true);
```

```
    $criteria->compare('hora_postulacion',$this->hora_postulacion,true);
```

```
    $criteria->compare('cedula_fun_recepcion',$this->cedula_fun_recepcion);
```

```
    $criteria->compare('des_datos_receptor',$this->des_datos_receptor,true);
```

```
    $criteria->compare('Cod_Jerarquia',$this->Cod_Jerarquia);
```

```
    $criteria->compare('id_asc_fecha',$this->id_asc_fecha);
```

```
    $criteria->compare('id_fecha_ingreso',$this->id_fecha_ingreso);
```

```
    $criteria->compare('num_modificar',$this->num_modificar);
```

```
    return new CActiveDataProvider($this, array(  
        'criteria'=>$criteria,
```

```
    ));
```

```
    }  
  
    /**  
    * Returns the static model of the specified AR class.
```

* Please note that you should have this exact method in all your CActiveRecord descendants!

* @param string \$className active record class name.

* @return Postulados the static model class

*/

public static function model(\$className=__CLASS__)

{

 return parent::model(\$className);

}

}